

Artificial Intelligence (AI) and Large Language Models (LLM)

Companion Guide v1.0

CIS Critical Security Controls v8.1

April 2026

Acknowledgments

The Center for Internet Security, Inc. (CIS®) would like to thank the many security experts who volunteer their time and talent to support the CIS Critical Security Controls® (CIS Controls®) and other CIS work. CIS products represent the effort of a veritable army of volunteers from across the industry, generously giving their time and talent in the name of a more secure online experience for everyone.

As a nonprofit organization driven by its volunteers, we are always in the process of looking for new topics and assistance in creating cybersecurity guidance. If you are interested in volunteering and/or have questions, comments, or have identified ways to improve this guide, please email us at: controlsinfo@cisecurity.org.

All references to tools or other products in this guide are provided for informational purposes only, and do not represent the endorsement by CIS of any particular company, product, or technology.

Principal Author

Jonathan Sander, Astrix

Editors

Andrew Dannenberger, CIS

Robin Regnier, CIS

Thomas Sager, CIS

Valecia Stocchetti, CIS

Contributors

Abhi Arikapudi, Databricks

Abhishek Iyer, Cybersecurity Leader

Andy Rivers, AWS

Christopher Misra, University of Massachusetts

Jack Zaldivar Jr., Databricks

Jeremy Pelegrin, University of Massachusetts

Geoff Hancock, Founder-CISO, Cyber Bridge Solutions

Michael Laing, Loblaw Companies Limited

Sharon Aby, Databricks

Tom Stryhn, GICSP, Cyber Security Engineer

This work is licensed under a Creative Commons Attribution-Non Commercial-No Derivatives 4.0 International Public License (the link can be found at <https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode>).

To further clarify the Creative Commons license related to the CIS Controls® content, you are authorized to copy and redistribute the content as a framework for use by you, within your organization and outside of your organization for non-commercial purposes only, provided that (i) appropriate credit is given to CIS, and (ii) a link to the license is provided. Additionally, if you remix, transform, or build upon the CIS Controls, you may not distribute the modified materials. Users of the CIS Controls framework are also required to refer to (<http://www.cisecurity.org/controls/>) when referring to the CIS Controls in order to ensure that users are employing the most up-to-date guidance. Commercial use of the CIS Controls is subject to the prior approval of the Center for Internet Security, Inc. (CIS).

Contents

Executive Summary	1
Scope	2
Methodology	4
How to Use This Guide	5
Glossary	7
Control 1: Inventory and Control of Enterprise Assets	9
AI LLM Applicability	9
Safeguards	10
Model Hosting and Deployment Considerations	11
Additional AI LLM Considerations	11
Control 2: Inventory and Control of Software Assets	12
AI LLM Applicability	12
Safeguards	12
Model Hosting and Deployment Considerations	13
Additional AI LLM Considerations	14
Control 3: Data Protection	15
AI LLM Applicability	15
Safeguards	15
Model Hosting and Deployment Considerations	17
Additional AI LLM Considerations	17
Control 4: Secure Configuration of Enterprise Assets and Software	18
AI LLM Applicability	18
Safeguards	18
Model Hosting and Deployment Considerations	20
Additional AI LLM Considerations	20
Control 5: Account Management	21
AI LLM Applicability	21
Safeguards	21
Model Hosting and Deployment Considerations	22
Additional AI LLM Considerations	23
Control 6: Access Control Management	24
AI LLM Applicability	24
Safeguards	24
Model Hosting and Deployment Considerations	25
Additional AI LLM Considerations	26
Control 7: Continuous Vulnerability Management	27
AI LLM Applicability	27
Safeguards	27
Model Hosting and Deployment Considerations	28
Additional AI LLM Considerations	29

Control 8: Audit Log Management	30
AI LLM Applicability	30
Safeguards	30
Model Hosting and Deployment Considerations	32
Additional AI LLM Considerations	32
Control 9: Email and Web Browser Protections	33
AI LLM Applicability	33
Safeguards	33
Model Hosting and Deployment Considerations	34
Additional AI LLM Considerations	35
Control 10: Malware Defenses	36
AI LLM Applicability	36
Safeguards	36
Model Hosting and Deployment Considerations	37
Additional AI LLM Considerations	37
Control 11: Data Recovery	38
AI LLM Applicability	38
Safeguards	38
Model Hosting and Deployment Considerations	39
Additional AI LLM Considerations	40
Control 12: Network Infrastructure Management	41
AI LLM Applicability	41
Safeguards	41
Model Hosting and Deployment Considerations	42
Additional AI LLM Considerations	43
Control 13: Network Monitoring and Defense	44
AI LLM Applicability	44
Safeguards	44
Model Hosting and Deployment Considerations	46
Additional AI LLM Considerations	46
Control 14: Security Awareness and Skills Training	47
AI LLM Applicability	47
Safeguards	47
Model Hosting and Deployment Considerations	48
Additional AI LLM Considerations	49
Control 15: Service Provider Management	50
AI LLM Applicability	50
Safeguards	50
Model Hosting and Deployment Considerations	51
Additional AI LLM Considerations	52
Control 16: Application Software Security	53
AI LLM Applicability	53
Safeguards	53
Model Hosting and Deployment Considerations	56
Additional AI LLM Considerations	56

Control 17: Incident Response Management	58
AI LLM Applicability	58
Safeguards	58
Model Hosting and Deployment Considerations	60
Additional AI LLM Considerations	60
Control 18: Penetration Testing	61
AI LLM Applicability	61
Safeguards	61
Model Hosting and Deployment Considerations	62
Additional AI LLM Considerations	63
Conclusion	64
Appendix A: CIS Controls	65
Appendix B: Acronyms and Abbreviations	66
Appendix C: Links and Resources	67

Executive Summary

As enterprises rapidly integrate Large Language Models (LLMs), Small Language Models (SLMs), and other generative Artificial Intelligence (AI) systems into business workflows and IT operations, these systems introduce security and operational risks that differ from traditional applications because they are probabilistic, prompt-driven, and often connected to retrieval systems, memory stores, and external tools that can take real actions. The primary attack surface shifts toward context integrity, tool misuse, data exposure, model-specific supply chain risks, and deterministically controlling probabilistic outputs. These differences require security teams to interpret existing best practices through an AI-aware lens to ensure that essential safeguards continue to provide strong coverage.

The CIS Critical Security Controls remain a globally trusted, prioritized set of defensive actions for reducing cybersecurity risk, but they were written before generative AI became an enterprise platform and parts of existing enterprise platforms. Many Safeguards map directly to AI-enabled systems (asset management, secure configuration, identity, logging, vulnerability management, and supplier governance). However, implementation must be interpreted through an AI-aware lens to address risks unique to LLM ecosystems, including direct and indirect prompt injection, retrieval poisoning, over-permissioned tool integrations, and provider-driven model updates. Applying the CIS Controls in this context therefore requires understanding how AI systems behave operationally, and where their risks diverge from traditional software or cloud services.

This guide adapts CIS Controls v8.1 for text-centric generative AI by translating the intent of each Control into practical expectations for AI-enabled systems across the full life cycle: training/fine-tuning, deployment, inference, monitoring, and retirement. It also highlights AI-specific risk domains that require explicit operational controls, including prompt and guardrail change control, context boundary enforcement, model and dataset provenance, and the containment levers required to respond quickly when AI-driven workflows behave unexpectedly.

By interpreting the CIS Controls through the lens of text-based generative AI, this guide provides practitioners with a practical, defensible way to secure emerging AI capabilities using the same prioritized framework already used across thousands of enterprises worldwide. The result is a consistent approach that strengthens the security of LLM- and SLM-enabled systems while preserving the flexibility to align with rapidly advancing AI technologies.

Scope

This guide provides targeted guidance for applying CIS Controls v8.1 to systems that use LLMs, SLMs, and other text-centric generative models. These technologies introduce operational and security characteristics not fully addressed by traditional software or cloud security guidance. This document interprets the CIS Controls in the context of text-based generative AI systems and highlights additional considerations needed to protect those systems effectively.

The scope includes systems that use LLMs, SLMs, or similar models for text or code generation, transformation, summarization, or analysis. It covers interactive applications, backend or Application Programming Interface (API)-driven use of models, embedded model components, and pipeline-based inference workflows.

Coverage extends across three primary deployment patterns: endpoint-hosted, enterprise-hosted, and SaaS-hosted models. For the purposes of this guide, the AI-enabled system is the primary unit of control, focusing on the technical components introduced exclusively to enable LLM operations. This guide applies to retrieval-augmented and memory-enhanced workflows only at the point where they intersect with the model's context window. While the external storage mechanisms (e.g., vector databases or document stores) are governed by traditional security frameworks, this guide focuses on the retrieved text as a primary security boundary and input vector.

Multimodal Capabilities and Non-Text Modalities

Modern foundation models increasingly accept non-text inputs, such as images, audio, and video. Enterprises must recognize that non-text inputs function as prompts. Just as a text prompt can contain malicious instructions, an image or audio file can contain hidden, embedded, or adversarial patterns — known as multimodal injections — that manipulate the model's behavior.

This guide governs non-text modalities specifically as input vectors for model reasoning.

- **In Scope:** Risks related to multimodal injection arise where an image or audio file, such as a screenshot containing hidden text, is used to hijack the model's context or bypass safety guardrails.
- **Out of Scope:** Risks related to the processing mechanics of these files (e.g., buffer overflows in image parsers) or domain-specific concerns such as biometric privacy (e.g., facial recognition policy) or deepfake generation.

Where non-text capabilities are enabled, enterprises must apply the AI LLM Best Practices in Control 15 (Service Provider Management), Control 16 (Application Software Security) and Control 18 (Penetration Testing) to these inputs with the same rigor applied to text.

Retrieval-Augmented Generation (RAG) and Scope Boundaries

Retrieval-Augmented Generation (RAG) allows models to access external data. While the underlying infrastructure, including vector databases, embeddings, and indexing jobs, is governed by the [Cloud Guide for CIS Controls v8.1](#) and [CIS Controls v8.1](#), this guide governs RAG as a critical input surface.

Since retrieved content is inserted directly into the model's context window, it functions as a "Trusted Input" and can be used for indirect prompt injection. Therefore, AI LLM best practices in this guide regarding input sanitization, data classification, and prompt hardening apply strictly to the retrieved text, regardless of where it is stored.

Topics Not Covered

The following areas are outside the scope of this guide, except where they intersect directly with the security and governance of text-centric model usage.

- Non-LLM and traditional machine learning models (e.g., tabular predictors, classical classifiers) that do not expose LLM-like conversational or generative interfaces are out of scope.
- Image, audio, video, and other non-text modalities, including image-generation systems, vision-only models, and image-related behaviors in multimodal models, are out of scope and require modality-specific controls not covered in this guide.
- Advanced agent orchestration, multi-step planning, and tool execution logic are out of scope for this guide but are addressed by the [AI Agents Companion Guide](#).
- Provider-internal model training infrastructure and hardware security beyond what is visible through contracts, configuration options, and observable model behavior are out of scope. Enterprises should address these via third-party risk management and service provider assessments.
- Fairness, bias, explainability, and ethics are important topics, but are not treated here, except where they intersect with abuse prevention, safety, and security controls.
- Human in the Loop (HITL) considerations are a very important safety, security, and governance control, but since this guide focuses on the LLM layer itself, and HITL would be embedded in the systems using those LLMs, it would fall outside the scope of this guide.
 - **Note:** LLM-enabled systems should generally not rely on model outputs or natural-language instructions as authorization to perform high-impact actions. HITL approval should be explicitly required when actions are irreversible, materially affect finances or legal obligations, change security posture, modify privileged configurations, initiate external communications, or write to authoritative systems of record. These boundaries must be enforced by application logic using identity, role-based access control, and workflow state, with approvals and denials logged for auditability.

Methodology

This guide follows the structure and intent of CIS Controls v8.1 and is designed to be read alongside the primary CIS Controls and the [Cloud Guide for CIS Controls v8.1](#). It does not introduce new top-level Controls or Safeguards; instead, it interprets each existing Control in the context of text-centric AI systems that use Large Language Models (LLMs), Small Language Models (SLMs), and related components.

For each CIS Control (1–18), this guide provides:

- **AI LLM Applicability:** How the Control applies to AI-enabled systems and why it matters.
- **Safeguards:** Practical guidance for adapting CIS Safeguards to AI contexts.
 - **Note:** “No Additional AI LLM Guidance” does not mean the Safeguard is irrelevant — it means the Safeguard as written already addresses AI LLM contexts without requiring additional guidance.
- **Model Hosting and Deployment Considerations:** What changes across endpoint-hosted, enterprise-hosted, and SaaS-hosted patterns.
- **Additional AI LLM Considerations:** Edge cases and tailoring guidance based on risk and architecture that can help enterprises tailor Safeguard implementations to their particular AI architectures and risk profile.

This guide also maintains the following design principles:

- **Layering with Related Companion Guides**

The [AI Agents Companion Guide](#) focuses on agentic, tool-using, and multi-step systems built on top of models. The [Model Context Protocol \(MCP\) Companion Guide](#) focuses on Model Context Protocol-based tool integration and associated security implications. This guide serves as the foundation for those two layers.
- **Model and Data Life Cycle Coverage**

The content in this guide collectively addresses the full life cycle of AI systems: data collection and preparation, training and fine-tuning, deployment, inference, monitoring, and retirement.
- **Risk-Based Tailoring**

Not all enterprises will deploy every type of model or feature. The guidance in this guide should be applied proportionally to risk, with higher rigor for systems that handle sensitive data, have high business impact, or expose powerful tools.

Enterprises should read each Control section in light of:

- Their selected Implementation Group(s)
- Their model hosting types (e.g., endpoint-hosted, enterprise-hosted, SaaS-hosted accessed via API)
- The presence or absence of RAG, memory, and tool invocation features

How to Use This Guide

[Implementation Groups \(IG1, IG2, IG3\)](#) continue to guide prioritization as with all preceding CIS Controls guides. Enterprises deploying LLMs in high-impact workflows may require Safeguards from higher Implementation Groups regardless of their overall IG classification.

This guide assumes that the enterprise has implemented certain CIS Controls appropriate to its operating environment. Specifically, it assumes:

- Asset and software inventory practices are in place (Controls 1 and 2)
- Secure configuration and configuration management processes exist (Control 4)
- Identity and access management is established and centrally managed (Controls 5 and 6)
- Logging, monitoring, and Security Information and Event Management (SIEM) capabilities exist and are actively maintained (Controls 8 and 13)
- A secure software development life cycle is in place (Control 16)
- Incident response processes are established and tested (Control 17)

This guide extends CIS Controls v8.1 by interpreting each Control in the context of text-centric AI systems. Enterprises using cloud infrastructure should also reference the [CIS Controls Cloud Companion Guide](#) for guidance on shared responsibility. Systems that use the Model Context Protocol (MCP) for tool and data integration should consult the [MCP Companion Guide](#) in addition to this document.

Model Hosting Types and Shared Responsibility

LLM-based systems can be deployed in several ways, each with different responsibility boundaries between the enterprise and its providers. This guide uses three model hosting types throughout:

- 1 Endpoint-Hosted Models**
- 2 Enterprise-Hosted Models**
- 3 SaaS-Hosted Models Accessed via API**

These hosting types should be understood as logical deployment patterns; a single enterprise may use more than one at the same time.

Endpoint-Hosted Models

Endpoint-hosted models run on user or developer endpoints such as laptops, workstations, or local servers, often under the control of a business unit or individual team.

Typical characteristics:

- Models, runtimes, and caches are stored locally
- Users may run small or distilled models for experimentation or offline use
- Local notebooks or Integrated Development Environment (IDE) integrations may embed LLM functionality

For endpoint-hosted models, the enterprise is broadly responsible for:

- Endpoint hardening, Endpoint Detection and Response (EDR)/anti-malware, and patch management
- Encryption, local storage, and log handling on the device
- Identity and access control for any credentials stored or used locally
- Governance of which models and tools may be installed on endpoints
- Ensuring that local AI tools do not bypass enterprise Data Loss Prevention (DLP), Secure Web Gateway (SWG), or proxy controls

Vendor responsibility, if any, may be limited to the model artifact or library itself (e.g., bug fixes, documented behavior). The enterprise still owns the runtime environment and data handling.

Enterprise-Hosted Models

Enterprise-hosted models run on infrastructure controlled by the enterprise, such as on-premises clusters, private cloud, or dedicated Virtual Private Cloud (VPC) in a public cloud provider.

Typical characteristics:

- Models and data reside on infrastructure under enterprise administrative control
- Graphics Processing Unit (GPU) clusters or high-memory Central Processing Unit (CPU) nodes are used for training and inference
- Internal Machine Learning Operations (MLOps) platforms manage model registries, pipelines, and deployments

For enterprise-hosted models, the enterprise is responsible for:

- Infrastructure security (e.g., network segmentation, operating system hardening, patching)
- Model storage, artifact integrity, and deployment processes
- Access control and identity management for model operations and data
- Logging, monitoring, and incident response across training and inference
- Configuration of model parameters, system prompts, tools, and guardrails
- Compliance with data residency, classification, and retention policies

Cloud infrastructure providers may still handle physical security and base cloud services, but the enterprise owns the security of workloads, models, and data within those services, as described in the [CIS Controls Cloud Companion Guide](#).

SaaS-Hosted Models Accessed via API

SaaS-hosted models are operated by a third-party provider and accessed via API or Software Development Kit (SDK). The provider controls the runtime, model instances, scaling, and many configuration options.

Typical characteristics:

- Models run entirely on provider infrastructure
- Enterprises interact through HTTPS APIs, SDKs, or provider-hosted user interfaces (UIs)
- Providers may offer built-in safety layers, logging, fine-tuning options, and tools

For SaaS-hosted models, the provider is generally responsible for:

- Physical and infrastructure security of their environment
- Model runtime security, patching, and internal monitoring
- Tenant isolation and basic model safety mechanisms

The enterprise remains responsible for:

- Data classification and deciding what data is allowed to be sent
- Identity, access control, and secret management for API keys and clients
- Integration logic, guardrails, and application behavior built around the model
- Contractual requirements for data retention, residency, and training reuse
- Monitoring and responding to suspicious usage from their identities
- DLP, SWG, and egress controls on outbound traffic to provider APIs

Shared Responsibility Themes

Across all hosting types:

- Models and prompts must be treated as high-sensitivity processing surfaces.
- Authorization and business logic remain the responsibility of the application and cannot be delegated to the model.
- RAG infrastructure, vector stores, and memory stores are typically under the enterprise's control and are governed by the core CIS Controls and the [CIS Controls Cloud Companion Guide](#), even when the model itself is SaaS-hosted.

Glossary

Adversarial Evaluation	Testing of AI systems using intentionally crafted inputs (prompts, documents, data) designed to elicit unsafe, unintended, incorrect, or policy-violating unexpected behavior.
AI-Bill of Materials (AI-BOM/Model BOM)	A structured record of models, dependencies, and associated components (e.g., frameworks, tokenizers, tools) used in a system, including version and provenance information.
AI Red-Teaming	Targeted testing of AI systems by internal or external experts using adversarial techniques specific to models, prompts, and AI-driven workflows.
Augmentation Store/Retrieval Corpus	The collection of documents, embeddings, or data sources used by retrieval-augmented systems to supply external context to models or agents.
Behavioral Drift	Unintended changes in model or agent behavior over time that are not explicitly intended and may introduce safety or reliability issues that tend to occur after updates, fine-tuning, prompt changes, or environmental shifts.
Concentration Risk (AI)	The risk that over-reliance on a single model, provider, or technology stack creates systemic operational, security, or resilience vulnerabilities.
Corpora	A large, structured collection of data used to train, evaluate, or fine-tune an AI model.
Data Leakage (Model Context)	Exposure of sensitive information through prompts, outputs, logs, embeddings, or model behavior, either accidentally or through adversarial activity.
Data Poisoning	Manipulation of training, fine-tuning, memory entries, or retrieval data to embed malicious or harmful behaviors into a model, the model's outputs, or agents.
Embedding/Embedding Model	A numeric vector representation of text (or other content) produced by a model, used for similarity search, retrieval, or clustering.
Endpoint-Hosted Model	A model running directly on user endpoints (e.g., laptops, workstations) under local or enterprise control.
Enterprise-Hosted Model	A model deployed on infrastructure controlled by the enterprise (on-premises, private cloud, or private VPC).
Fine-Tuning	Additional training of a base model on task- or domain-specific data to adapt its behavior without full retraining from scratch.
Guardrail	Application or middleware logic that constrains or validates model and agent behavior, inputs, outputs, or tool actions, enforcing business rules and safety policies outside the model.
Implementation Group (IG)	Grouped IG1, IG2, and IG3, these are a way for enterprises to prioritize the implementation of the CIS Controls.
Inference	Phase of an AI system in which a deployed model processes input data (e.g., prompts, retrieved context, or multimodal inputs) to generate outputs via a probabilistic estimation, without modifying its trained parameters. Unlike traditional deterministic software, inference in LLMs involves calculating the statistical likelihood of sequences (e.g., next-token prediction) to produce a response without modifying the model's underlying trained parameters.
Jailbreak	An attempt to circumvent model safety controls or policies using crafted prompts or adversarial context.
Kill Switch	A control or mechanism that allows rapid disabling of a model, endpoint, tool capability, agent, or entire AI subsystem in response to an incident.
Large Language Model (LLM)	A generative model with a large number of parameters, designed to process and produce natural language (and often code).
Memory (Short-Term/Long-Term)	Context maintained by a system across interactions, either transiently (short-term, within a session) or persisted (long-term) to influence future model behavior.
Model Card/System Card	Documentation describing a model's capabilities, limitations, training data characteristics, intended use cases, and known risks.
Model Context/Context Window	The range of tokens (input and/or prior output) that the model can attend to when generating a response.

Model Extraction	Attempts to replicate or approximate a proprietary model's behavior (or underlying parameters) by querying it at scale and analyzing responses.
Model Hosting Type	The deployment category describing where and how a model runs: endpoint-hosted, enterprise-hosted, or SaaS-hosted accessed via API.
Model Provenance	The origin, lineage, and transformation history of a model, including base model, fine-tuning datasets, and training processes.
Model Registry	A centralized repository used to store, version, and track machine learning models. It functions as the primary software inventory for AI assets, tracking them as they are updated and deployed.
Non-Text Modality/Non-Text Token	Inputs or outputs such as images, audio, video, or the token representations of such non-text content in multimodal models.
Poisoning (RAG/Retrieval)	Introducing malicious or misleading content into retrieval corpora or vector stores so that it influences model behavior in unintended ways.
Prompt	The input content provided to a model or agent, including instructions, questions, or data examples.
Prompt Injection	An attack where malicious instructions are embedded in content processed by an LLM to manipulate its behavior. Indirect prompt injection occurs when hostile instructions arrive through retrieved resources, tool outputs, or other external content rather than direct user input.
Retrieval-Augmented Generation (RAG)	An architecture where external data is retrieved (e.g., via embeddings and vector search) and then incorporated into the model or agent context as additional input.
Retrieved Context/Retrieval Corpus	The specific documents or text chunks selected by retrieval mechanisms and passed into the model as context.
SaaS-Hosted Model	A model operated by a third-party provider and exposed over API/SDK or web interface, where runtime and infrastructure are managed by the provider.
Secure Web Gateway (SWG)	A control that inspects and governs web traffic, often providing URL filtering, content inspection, and DLP.
Shadow AI	Unapproved or unmanaged AI tools, accounts, agents, or services within an enterprise, such as personal accounts on public AI services or unsanctioned model deployments.
Small Language Model (SLM)	A smaller, more resource-efficient language model, typically used for constrained devices, specific tasks, or cost-sensitive deployments.
System Prompt	A privileged, often hidden, instruction block that sets a model's or agent's overall behavior, tone, or policy.
Token	A discrete unit (subword, character, or other encoded element) used by models to represent text or other content internally.
Tokenization	The process of converting input text (or other content) into tokens that a model can process.
Tool (Model-Level Tool/Function Call)	An external capability an agent invokes, such as APIs, databases, execution engines, browsers, file handlers, or custom actions.
User Prompt	Input provided by an end user or calling application to request an action from a model or agent.
Vector Store/Vector Database	A storage system optimized for similarity search over embeddings, typically used to implement retrieval in RAG systems.

Control 1: Inventory and Control of Enterprise Assets

Actively manage (inventory, track, and correct) all enterprise assets (end-user devices, including portable and mobile; network devices; non-computing/Internet of Things (IoT) devices; and servers) connected to the infrastructure physically, virtually, remotely, and those within cloud environments, to accurately know the totality of assets that need to be monitored and protected within the enterprise. This will also support identifying unauthorized and unmanaged assets to remove or remediate.

AI LLM Applicability

LLM systems rely on specialized compute infrastructure, including GPUs, inference hosts, high-memory CPU nodes, and sometimes unmanaged endpoints such as data scientists' laptops or personal cloud instances. These environments are frequently created dynamically, scaled automatically, or operated by research teams outside normal Information Technology (IT) pathways.

LLM-related assets often include:

- Endpoint-hosted models running locally (e.g., on analyst/developer machines)
- Enterprise-hosted infrastructure such as GPU clusters, inference servers, and model hosting platforms
- Shadow AI compute such as personal VMs or unauthorized cloud instances
- Storage systems that hold model artifacts, cached prompts, embeddings, or evaluation outputs

Some data and systems are included here because they should be included in a robust inventory, even if they are not in scope for these controls specifically. These include:

- Model endpoints and gateways (API gateways, LLM proxies, egress proxies)
- Vector databases/embedding stores/retrieval indexes
- Model registries and artifact stores (weights, fine-tunes, adapters)
- Prompt/config stores (system prompts, templates, policies)
- Identity artifacts: service accounts, workload identities, API keys tied to model access
- Monitoring assets: prompt/output logs, evaluation datasets, red-team harnesses

AI systems process sensitive, high-value data, so inventory blind spots can lead to gaps in data protection, access control, and monitoring. Maintain a complete inventory, including endpoints running small local models that can store or process sensitive prompts, logs, and temporary outputs, to enable consistent secure configuration, patching, segmentation, and monitoring across the AI environment. When AI infrastructure is created outside standard IT pathways, these exceptions should require tagging, logging, and owner attribution, and still offer auto-quarantine/disable features.

Safeguards

CIS Control 1: Inventory and Control of Enterprise Assets

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
1.1	Establish and Maintain Detailed Enterprise Asset Inventory	Establish and maintain an accurate, detailed, and up-to-date inventory of all enterprise assets with the potential to store or process data, to include: end-user devices (including portable and mobile), network devices, non-computing/IoT devices, and servers. Ensure the inventory records the network address (if static), hardware address, machine name, enterprise asset owner, department for each asset, and whether the asset has been approved to connect to the network. For mobile end-user devices, MDM type tools can support this process, where appropriate. This inventory includes assets connected to the infrastructure physically, virtually, remotely, and those within cloud environments. Additionally, it includes assets that are regularly connected to the enterprise's network infrastructure, even if they are not under control of the enterprise. Review and update the inventory of all enterprise assets bi-annually, or more frequently.	●	●	●	Maintain an inventory of all AI LLM enterprise assets, including compute (e.g., GPUs/inference/training), control plane (e.g., model gateways, registries, prompt/config stores), data plane (e.g., vector databases, retrieval corpora, log stores), and integration assets (e.g., API proxies, SDK clients, service identities). Record owner, environment (development/staging/production), hosting type, data sensitivity handled, and approved use case. Note: Some data and systems are included here because they should be included in a robust inventory even if they are not in scope for these controls specifically.
1.2	Address Unauthorized Assets	Ensure that a process exists to address unauthorized assets on a weekly basis. The enterprise may choose to remove the asset from the network, deny the asset from connecting remotely to the network, or quarantine the asset.	●	●	●	Identify and monitor unapproved or "shadow AI" environments such as personal laptops, unmanaged cloud resources, and unauthorized VMs running local models. Experimentation by analysts, developers, or researchers often leads to unmanaged environments. Shadow assets handling sensitive data create unmanaged risk and must be identified and brought under governance or decommissioned.
1.3	Utilize an Active Discovery Tool	Utilize an active discovery tool to identify assets connected to the enterprise's network. Configure the active discovery tool to execute daily, or more frequently.		●	●	Ensure that endpoint-hosted models are visible to Mobile Device Management (MDM)/EDR systems and included in existing asset governance processes. Ensure that detection includes local model runners and toolchains (e.g., Ollama/LM Studio/llama.cpp, local vector DBs, notebook runtimes, containerized inference). Local LLMs can store cached prompts, embeddings, or logs containing sensitive data. Including endpoints in standard asset management tools ensures centralized enforcement of configuration, encryption, monitoring, and response policies. Discover cloud AI assets via cloud asset inventory (accounts/projects), GPU instance types, managed AI services, and container registries.
1.4	Use Dynamic Host Configuration Protocol (DHCP) Logging to Update Enterprise Asset Inventory	Use DHCP logging on all DHCP servers or Internet Protocol (IP) address management tools to update the enterprise's asset inventory. Review and use logs to update the enterprise's asset inventory weekly, or more frequently.		●	●	No Additional AI LLM Guidance
1.5	Use a Passive Asset Discovery Tool	Use a passive discovery tool to identify assets connected to the enterprise's network. Review and use scans to update the enterprise's asset inventory at least weekly, or more frequently.			●	Focus passive discovery on parsing application and cloud logs for model interaction signatures, since AI software assets rarely generate the distinct network artifacts captured by traditional DHCP listeners. LLMs, especially LLMs used through APIs from major SaaS providers of AI services, can be a data protection risk and may only show up in their interactions with other systems since they are changing so fast that active discovery may not have methods to probe them.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local machines may run small LLMs that bypass centralized infrastructure entirely.
- MDM/EDR must detect and manage these assets, including model binaries, local cache directories, and storage of embeddings or logs.

Enterprise-Hosted Models

- GPU clusters, inference servers, and fine-tuning environments must appear in centralized Configuration Management Database (CMDB) or asset repositories.
- Dynamic or ephemeral training/inference workloads (e.g., auto-scaled nodes) require automated discovery mechanisms.

SaaS-Hosted Models Accessed via API

- While compute is provider-managed, integration assets, including gateways, API proxies, client libraries, credentials, and logging endpoints, must be inventoried.
- Identify which internal systems interact with SaaS model endpoints and treat those as part of the AI asset landscape.
- Identify what internal data is sent to the model, assess its sensitivity, and review the permissions of any service accounts, tokens, tools, or plugins used in integrations.

Additional AI LLM Considerations

- Storage for model inputs/outputs (e.g., logs, cached prompts, and embeddings) may reside in multiple runtime directories or cloud buckets; these must be tracked as assets.
- Research teams frequently deploy ad-hoc environments for experimentation. These must be discoverable or prohibited depending on policy.
- Auto-scaling inference environments can create short-lived compute instances. Asset inventory tools must capture and tag them before they are destroyed.
- Bring-your-own-model (BYOM) workflows introduce third-party model artifacts requiring separate tracking and validation.
- Asset inventories supporting AI LLM systems must maintain strict data quality and consistency (e.g., naming, versioning, metadata, ownership). Even minor discrepancies can lead models or automation to infer equivalence or continuity, creating compounding assumptions and undermining control reliability.

Control 2: Inventory and Control of Software Assets

Actively manage (inventory, track, and correct) all software (operating systems and applications) on the network so that only authorized software is installed and can execute, and that unauthorized and unmanaged software is found and prevented from installation or execution.

AI LLM Applicability

LLM systems involve a wide collection of software assets beyond the models themselves. These include model artifacts, inference runtimes, training and fine-tuning pipelines, embedding models, orchestration frameworks, SDKs, client libraries, and utilities used for evaluation or preprocessing. Many AI components, such as tokenizers, vector databases, or fine-tuning toolchains, can be installed by research teams outside normal governance, making a complete inventory essential.

Since LLM-based systems evolve quickly, model versions, serving frameworks, and dependencies may change without the visibility traditional IT asset systems expect. SaaS-hosted models also introduce “virtual” software assets, where the enterprise does not control the underlying runtime but must track the specific models, versions, and APIs in use. A complete and accurate record of all model-related software is critical for understanding exposure to vulnerabilities, deprecated model versions, insecure libraries, or compromised supply-chain components.

Model and dataset integrity is a core security requirement. Without strong software asset inventory, enterprises cannot verify that a deployed model corresponds to an approved version, that the underlying libraries are supported and patched, or that model-serving processes have not been unintentionally or maliciously altered.

Safeguards

CIS Control 2: Inventory and Control of Software Assets

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
2.1	Establish and Maintain a Software Inventory	Establish and maintain a detailed inventory of all licensed software installed on enterprise assets. The software inventory must document the title, publisher, initial install/use date, and business purpose for each entry; where appropriate, include the Uniform Resource Locator (URL), app store(s), version(s), deployment mechanism, decommission date, and number of licenses. Review and update the software inventory bi-annually, or more frequently.				Maintain an inventory of all models and AI-related software components, including model name, version, license, source, and hosting type. This inventory must include base models, fine-tuned variants, model registry exports, container registry manifests, tokenizers, embedding models, SBOM/MBOM, SDKs with versions, and runtimes. Recording model origins and licensing terms is essential for compliance and risk assessment.
2.2	Ensure Authorized Software is Currently Supported	Ensure that only currently supported software is designated as authorized in the software inventory for enterprise assets. If software is unsupported, yet necessary for the fulfillment of the enterprise’s mission, document an exception detailing mitigating controls and residual risk acceptance. For any unsupported software without an exception documentation, designate as unauthorized. Review the software list to verify software support at least monthly, or more frequently.				Treat model weights (e.g., Llama, Mistral) as software artifacts that require a clear line of support to be considered authorized for use. This support must be established through either a commercial vendor contract or a formal internal enterprise support program that has technically and financially evaluated specific open-weight or OSS models. Commercial SaaS agreements typically do not extend to self-hosted open models (e.g., Google’s Gemma or OpenAI’s gpt-oss series), even if the open model comes from the vendor you have the commercial agreement with. Models lacking a designated external or internal support path are considered unsupported; relying on these artifacts leaves the enterprise vulnerable to unpatched security flaws, performance degradation, and sudden loss of compatibility with inference runtimes.

CIS Control 2: Inventory and Control of Software Assets

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
2.3	Address Unauthorized Software	Ensure that unauthorized software is either removed from use on enterprise assets or receives a documented exception. Review monthly, or more frequently.	●	●	●	Remove unapproved or unsupported AI components from environments where they are not authorized. Untracked third-party models, libraries, or frameworks introduce supply-chain and operational risk. Removing them ensures only approved, secure components are used.
2.4	Utilize Automated Software Inventory Tools	Utilize software inventory tools, when possible, throughout the enterprise to automate the discovery and documentation of installed software.		●	●	Use automated tools to track inference servers, runtimes, fine-tuning libraries, and embedding models used in production, development, and research environments. These components are often updated independently of the models they serve, and untracked updates may introduce vulnerabilities or break compatibility.
2.5	Allowlist Authorized Software	Use technical controls, such as application allowlisting, to ensure that only authorized software can execute or be accessed. Reassess bi-annually, or more frequently.		●	●	Validate ML frameworks and runtime environments to prevent unsafe loading or evaluation of untrusted code or model extensions. Some AI frameworks auto-discover operations or extensions; without validation, attackers may introduce malicious components or achieve arbitrary code execution.
2.6	Allowlist Authorized Libraries	Use technical controls to ensure that only authorized software libraries, such as specific .dll, .ocx, and .so files, are allowed to load into a system process. Block unauthorized libraries from loading into a system process. Reassess bi-annually, or more frequently.		●	●	Allowlist/load-control for runtime libraries (CUDA, cuDNN, PyTorch extensions, kernel modules, native deps) and block unknown shared objects in model-serving processes.
2.7	Allowlist Authorized Scripts	Use technical controls, such as digital signatures and version control, to ensure that only authorized scripts, such as specific .ps1 and .py files, are allowed to execute. Block unauthorized scripts from executing. Reassess bi-annually, or more frequently.			●	No Additional AI LLM Guidance

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local installations of model files, tokenizers, libraries, and vector stores may bypass centralized governance.
- Inventory systems must detect and track local ML packages, Python environments, or container runtimes that enable local inference.

Enterprise-Hosted Models

- GPU clusters and internal Machine Learning Operations (MLOps) platforms may install or update software automatically. Inventories must integrate with Continuous Integration/Continuous Delivery (CI/CD), model registries, and environment build pipelines.
- Fine-tuning or training environments often rely on specialized libraries (e.g., PyTorch, TensorFlow, Compute Unified Device Architecture (CUDA) drivers) that require visibility and tracking.

SaaS-Hosted Models Accessed via API

- Enterprises must inventory:
 - Which SaaS model endpoints are used
 - Which model versions or IDs are referenced
 - Which SDK versions client systems rely on
- Provider runtime updates are outside enterprise control. Versioning and compatibility must be documented to prevent unexpected behavior.

Additional AI LLM Considerations

- Tokenizers and embedding models change over time, and mismatched versions can produce inconsistent retrieval or prompt behavior.
- Some open-source models include custom or experimental operators that may require special runtime support, and these must be tracked explicitly.
- Evaluation datasets and prompt-test harnesses may also be treated as software assets if they drive system behavior.
- Third-party agent frameworks, orchestration tools, and model routers, even if used lightly, must be inventoried to avoid supply-chain compromise.
- Dependencies specific to Graphics Processing Units (GPUs) (e.g., Compute Unified Device Architecture (CUDA)/CUDA Deep Neural Network (cuDNN) library, and vendor drivers) must be considered part of the AI software stack.

Control 3: Data Protection

Develop processes and technical controls to identify, classify, securely handle, retain, and dispose of data.

AI LLM Applicability

LLM-based systems introduce new categories of data that must be protected with the same rigor as traditional structured and unstructured data. Prompts, completions, embeddings, fine-tuning datasets, RAG-retrieved text, memory, and logs often contain sensitive information even when not explicitly labeled as such.

Because LLMs operate in natural language, inputs may unintentionally include regulated or confidential material. Likewise, outputs, such as completions or embeddings, may contain transformed sensitive data, reconstruction of prior inputs, or sensitive information inferred from context. LLM interaction logs, often retained for debugging or monitoring, can become high-sensitivity records that must be governed accordingly.

Training and fine-tuning datasets are particularly sensitive. Their unauthorized modification introduces risks such as data poisoning or embedded malicious behavior that persists across model versions. Likewise, retrieval sources used in RAG systems can mix trusted and untrusted content unless intentionally segmented and validated.

Enterprises must define how each type of LLM-related data is classified, stored, transmitted, retained, and destroyed, and must ensure that model-internal mechanisms such as memory or retrieved context do not become uncontrolled data repositories. This also includes ensuring that LLMs which run in specific classification contexts are only exposed to data in that classification context; for example, an LLM that is only cleared to handle public data should not process Material Non-Public Information (MNPI). Non-text modalities, when enabled, expand these concerns further and may exceed the coverage of this guide.

Safeguards

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
3.1	Establish and Maintain a Data Management Process	Establish and maintain a documented data management process. In the process, address data sensitivity, data owner, handling of data, data retention limits, and disposal requirements, based on sensitivity and retention standards for the enterprise. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Enforce data minimization and integrity controls across the AI life cycle, sanitizing sensitive inputs before inference or training while securing fine-tuning datasets and RAG corpora against unauthorized modification. Models can memorize sensitive data or inherit malicious behaviors from tainted inputs; rigorous management prevents irreversible data leakage and defends against "data poisoning" attacks where adversaries embed backdoors or bias directly into the model's logic.
3.2	Establish and Maintain a Data Inventory	Establish and maintain a data inventory based on the enterprise's data management process. Inventory sensitive data, at a minimum. Review and update inventory annually, at a minimum, with a priority on sensitive data.	●	●	●	Inventory all data that has been fed into LLMs, especially SaaS-hosted LLMs, as prompts, attachments, or using any other methods (including data that may come from RAG processes, vector databases, etc.), and ensure all this data is properly tracked and annotated in the enterprise data inventory. LLMs processing enterprise data, especially sensitive data that may have regulatory or legal implications, has known and unknown risks that must be accounted for. This is especially true for SaaS-hosted LLMs, which may use such data for model training.

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
3.3	Configure Data Access Control Lists	Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●	Retrieved content and long-term memory act as “trusted inputs” that can bypass other controls; unauthorized access or poisoning of these stores allows attackers to manipulate model behavior, exfiltrate sensitive knowledge, or permanently embed malicious instructions. Enforce strict filtering and sanitization on all content retrieved for the model context window from sources such as AI context repositories, including vector databases, retrieval indexes, and persistent memory stores. Ensure that retrieved text is governed by the same classification rigor as the original source before it is provided as an input vector.
3.4	Enforce Data Retention	Retain data according to the enterprise's documented data management process. Data retention must include both minimum and maximum timelines.	●	●	●	Apply retention policies to model-related artifacts. Establish and enforce documented retention policies for model interaction logs, embeddings, cached outputs, and other transient artifacts to ensure storage duration is intentional, minimal, and compliant with regulations and internal requirements.
3.5	Securely Dispose of Data	Securely dispose of data as outlined in the enterprise's documented data management process. Ensure the disposal process and method are commensurate with the data sensitivity.	●	●	●	Apply secure disposal policies to model-related artifacts. Implement secure deletion processes for model interaction logs, embeddings, cached outputs, and transient artifacts to ensure sensitive or residual content is irreversibly removed when no longer required.
3.6	Encrypt Data on End-User Devices	Encrypt data on end-user devices containing sensitive data. Example implementations can include: Windows BitLocker®, Apple FileVault®, Linux® dm-crypt.	●	●	●	Ensure that all endpoints used for AI work are using encrypted storage to ensure all the LLM assets on such endpoints (e.g., local prompt/output caches and local vector stores) are properly protected and require MDM controls to enforce this.
3.7	Establish and Maintain a Data Classification Scheme	Establish and maintain an overall data classification scheme for the enterprise. Enterprises may use labels, such as “Sensitive,” “Confidential,” and “Public,” and classify their data according to those labels. Review and update the classification scheme annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Classify LLM-related data (e.g., prompts, completions, embeddings, datasets) according to existing enterprise data classification schemes. Proper classification ensures sensitive information is handled according to enterprise requirements, including restrictions on storage, sharing, retention, and access. LLM-related data should not be treated as ephemeral simply because it originates from a generative model.
3.8	Document Data Flows	Document data flows. Data flow documentation includes service provider data flows and should be based on the enterprise's data management process. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Track network segments and storage systems used to host or serve models, including specialized hardware, inference clusters, dedicated GPU nodes, and model artifact storage. Model data and runtime state may reside across distributed storage systems and network zones. Mapping these ensures proper segmentation, backup, access control, and monitoring.
3.9	Encrypt Data on Removable Media	Encrypt data on removable media.		●	●	No Additional AI LLM Guidance
3.10	Encrypt Sensitive Data in Transit	Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●	Encrypt sensitive model-related data, including datasets, embeddings, outputs, and intermediate representations, while in transit to protect confidentiality and integrity during network transfer.
3.11	Encrypt Sensitive Data at Rest	Encrypt sensitive data at rest on servers, applications, and databases. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●	Encrypt all model-related data at rest and enforce the use of secure, OS-managed storage for all API credentials and access tokens to prevent plaintext exposure. Unprotected artifacts, including datasets, embeddings, and caches, can leak sensitive semantic information; likewise, exposed API keys allow attackers to easily gain unauthorized access to costly SaaS models and private inference endpoints.

CIS Control 3: Data Protection

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
3.12	Segment Data Processing and Storage Based on Sensitivity	Segment data processing and storage based on the sensitivity of the data. Do not process sensitive data on enterprise assets intended for lower sensitivity data.				Enforce strict network and storage segmentation between AI training environments, inference pipelines, and retrieval corpora, isolating high-trust model components from untrusted external inputs and general enterprise networks. Without explicit trust boundaries, low-trust external content (like emails) can contaminate core knowledge bases, and compromised training pipelines can be used to inject backdoors; segmentation limits the blast radius and prevents lateral movement into sensitive model systems.
3.13	Deploy a Data Loss Prevention Solution	Implement an automated tool, such as a host-based Data Loss Prevention (DLP) tool to identify all sensitive data stored, processed, or transmitted through enterprise assets, including those located onsite or at a remote service provider, and update the enterprise's data inventory.				Ensure that DLP solutions are examining data being passed to and from LLMs of all types to ensure that data flow is following enterprise policies. LLM interactions represent just as large a vector for data loss as any other application or network data flow and must be given as much attention as these to reduce risk. Assume that DLP and inspection controls may not fully parse image, audio, or video content and require compensating controls or restrictions on these modes.
3.14	Log Sensitive Data Access	Log sensitive data access, including modification and disposal.				Monitor all LLM logging for indications of access to known sensitive data.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local prompts, logs, and model caches may sit unencrypted on user machines; strong endpoint encryption and MDM enforcement are essential.
- External content ingestion (e.g., local files, screenshots) must be governed or restricted.

Enterprise-Hosted Models

- Training datasets, embeddings, and augmentation stores reside in enterprise-controlled systems; segmentation between environments (training, staging, inference) reduces poisoning risk.
- Storage systems (object stores, feature stores, vector databases) require explicit roles and least privilege controls.

SaaS-Hosted Models Accessed via API

- Enterprises must rely on provider assurances for at-rest encryption, retention, explicit deletion guarantees, subsequent deletion timelines, and training opt-out and must classify which data is permitted to be sent.
- Prompt and completion logs often remain on provider infrastructure; contractual obligations must define residency and retention.

Additional AI LLM Considerations

- Non-text tokens (images, audio) introduce new leakage and steganography risks not covered under the AI LLM Applicability above and should be treated as separate risk surfaces.
- Embeddings can leak sensitive information even when raw text is not stored; embedding stores must be classified and protected accordingly.
- Memory and RAG pipelines can create implicit, accumulated data stores. Without explicit governance, they become unsanctioned sensitive data repositories.
- If model outputs are fed downstream into other applications, ensure that no sensitive data propagates unintentionally into logs, user interfaces, or unprotected subsystems.

Control 4: Secure Configuration of Enterprise Assets and Software

Establish and maintain the secure configuration of enterprise assets (end-user devices, including portable and mobile; network devices; non-computing/IoT devices; and servers) and software (operating systems and applications).

AI LLM Applicability

LLM systems introduce unique configuration surfaces that extend beyond those of traditional applications. In addition to server, container, and OS configurations, enterprises must also govern:

- Model-level configuration such as temperature, max tokens, system prompts, tool enablement, and model routing behavior.
- Training/fine-tuning environments, which integrate high-privilege data handling with complex, dependency-heavy toolchains.
- Inference-serving infrastructure, which may include GPUs, specialized runtimes, container images, and frameworks that load unverified artifacts by default.
- Endpoint-hosted LLMs, which behave like local applications but may process sensitive data and store caches or outputs unencrypted.
- Tooling and plugins, which add execution paths (like code interpreters or function-calling tools) that must be explicitly controlled.

LLM systems are highly sensitive to configuration drift. Small changes, such as altering a system prompt, enabling a new capability, or modifying token limits, can materially change system behavior. Some providers update SaaS model versions automatically, so integrations must avoid configurations that inadvertently adopt new model behavior without review. Secure configuration must therefore include explicit version pinning, controlled prompt and tool-management workflows, and hardened execution environments for all AI-adjacent components.

Safeguards

CIS Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
4.1	Establish and Maintain a Secure Configuration Process	Establish and maintain a documented secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.				Establish managed configuration baselines for all AI components, enforcing version control for system prompts and model parameters, strict version pinning for model artifacts, and hardened defaults for inference environments. System prompts, token limits, and decoding parameters define model behavior as critical as any compiled code; treating them as unmanaged text leads to silent behavioral drift or security bypasses. Without strict version pinning (avoiding "latest" tags), enterprises risk implicit upgrades that introduce new vulnerabilities or break existing guardrails. Furthermore, hardened baselines for serving environments, including rate limits and authentication, are essential to ensure consistency and prevent model extraction or denial-of-service attacks from abusive traffic patterns.
4.2	Establish and Maintain a Secure Configuration Process for Network Infrastructure	Establish and maintain a documented secure configuration process for network devices. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.				No Additional AI LLM Guidance

CIS Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
4.3	Configure Automatic Session Locking on Enterprise Assets	Configure automatic session locking on enterprise assets after a defined period of inactivity. For general purpose operating systems, the period must not exceed 15 minutes. For mobile end-user devices, the period must not exceed 2 minutes.	●	●	●	No Additional AI LLM Guidance
4.4	Implement and Manage a Firewall on Servers	Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●	Configure host-based firewalls on model hosting servers to strictly limit inbound inference requests to authorized internal gateways and block all unauthorized outbound connections to public repositories where such connections are not pre-authorized. Compromised model runtimes often attempt to establish reverse shells or exfiltrate training data; strict egress filtering prevents attackers from moving stolen IP to external command-and-control servers.
4.5	Implement and Manage a Firewall on End-User Devices	Implement and manage a host-based firewall or port-filtering tool on end-user devices, with a default-deny rule that drops all traffic except those services and ports that are explicitly allowed.	●	●	●	Configure host-based firewalls on end-user devices to block inbound connections to local AI runtimes and restrict outbound traffic to only authorized model APIs and repositories. Local AI tools often bind API services to open network ports by default; without firewall restrictions, attackers on the same network can query private models or exploit runtime vulnerabilities.
4.6	Securely Manage Enterprise Assets and Software	Securely manage enterprise assets and software. Example implementations include managing configuration through version-controlled Infrastructure-as-Code (IaC) and accessing administrative interfaces over secure network protocols, such as Secure Shell (SSH) and Hypertext Transfer Protocol Secure (HTTPS). Do not use insecure management protocols, such as Telnet (Teletype Network) and HTTP, unless operationally essential.	●	●	●	Enforce secure management practices for all AI execution environments, applying strict sandboxing and network isolation to endpoint-hosted models while mandating centralized, scoped secret management for SaaS API credentials. Local LLM runtimes effectively turn endpoints into servers that cache sensitive data and write to disk; without rigorous sandboxing and network constraints, they become open pathways for data leakage or lateral movement.
4.7	Manage Default Accounts on Enterprise Assets and Software	Manage default accounts on enterprise assets and software, such as root, administrator, and other pre-configured vendor accounts. Example implementations can include: disabling default accounts or making them unusable.	●	●	●	No Additional AI LLM Guidance
4.8	Uninstall or Disable Unnecessary Services on Enterprise Assets and Software	Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.		●	●	Disable all tools, plugins, and non-text modalities by default; enable only with explicit approval. Tools and plugins empower models to execute real-world actions that can bypass standard application policies, while non-text modalities (e.g., images, audio) introduce complex attack vectors, like visual jailbreaks or steganography, that exceed the capabilities of standard text-based safety filters. Enabling these features without review opens the door to unauthorized command execution and unmonitored data processing. A default-deny posture ensures that these high-risk capabilities are only active when the specific security controls required to protect them are confirmed to be in place.
4.9	Configure Trusted DNS Servers on Enterprise Assets	Configure trusted DNS servers on network infrastructure. Example implementations include configuring network devices to use enterprise-controlled DNS servers and/or reputable externally accessible DNS servers.		●	●	Use enterprise DNS and protect resolution paths for model endpoints, model registries, and dependency sources. Alert on suspicious AI-related domains.
4.10	Enforce Automatic Device Lockout on Portable End-User Devices	Enforce automatic device lockout following a predetermined threshold of local failed authentication attempts on portable end-user devices, where supported. For laptops, do not allow more than 20 failed authentication attempts; for tablets and smartphones, no more than 10 failed authentication attempts. Example implementations include Microsoft® InTune Device Lock and Apple® Configuration Profile maxFailedAttempts.		●	●	No Additional AI LLM Guidance
4.11	Enforce Remote Wipe Capability on Portable End-User Devices	Remotely wipe enterprise data from enterprise-owned portable end-user devices when deemed appropriate such as lost or stolen devices, or when an individual no longer supports the enterprise.		●	●	No Additional AI LLM Guidance

CIS Control 4: Secure Configuration of Enterprise Assets and Software

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
4.12	Separate Enterprise Workspaces on Mobile End-User Devices	Ensure separate enterprise workspaces are used on mobile end-user devices, where supported. Example implementations include using an Apple® Configuration Profile or Android™ Work Profile to separate enterprise applications and data from personal applications and data.				Enforce separation of enterprise AI data and personal AI usage on mobile devices using containerization or work profiles. This prevents sensitive corporate data from leaking into personal AI accounts or unmanaged model interactions on employee devices, while also protecting enterprise data from personal device compromises.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local LLM clients require secure configuration baselines just like any endpoint software. Uncontrolled plugins or local “tools” must be disabled or governed centrally.
- Local filesystem permissions and sandboxing are critical because models may write intermediate files.

Enterprise-Hosted Models

- GPU/CPU inference servers and training clusters require hardened base images and controlled update pathways.
- Infrastructure-as-code may configure token limits, API gateways, or routing policies; these must be version-controlled.
- Auto-scaling inference platforms must ensure new nodes inherit secure baselines automatically.

SaaS-Hosted Models Accessed via API

- Enterprises must verify and configure:
 - History retention
 - Content logging
 - Model versioning or “version freeze” options
 - Tool enablement
 - API capability scopes
- Provider-side updates may alter behavior; strict version pinning and compatibility testing are essential.

Additional AI LLM Considerations

- System prompts are configuration, not content; treat them like code with approvals and documentation.
- Decoding parameters (temperature, Top-p sampling, etc.) affect safety and consistency; unauthorized changes must be prevented.
- Some model variants include specialized behaviors (reasoning modes, extended context). Enabling these may expand the risk surface.
- Tools, such as browsers, code interpreters, and search APIs, often run in separate execution environments, and configuration drift in those can undermine model governance.
- If non-text modalities are present, configurations must allow them to be disabled, inspected, or routed through additional controls.

Control 5: Account Management

Use processes and tools to assign and manage authorization to credentials for user accounts, including administrator accounts, as well as service accounts, to enterprise assets and software.

AI LLM Applicability

LLM systems rely heavily on non-human identities, such as service accounts, API keys, OAuth clients, Identity and Access Management (IAM) roles, and workload identities, to perform tasks such as training, fine-tuning, deployment, inference serving, data retrieval, and evaluation. These identities typically have access to:

- Sensitive training datasets
- Model artifacts
- Fine-tuning pipelines
- SaaS model APIs and billing surfaces
- Logging and monitoring endpoints
- High-value storage systems, such as vector databases or embeddings stores

Without careful oversight, service accounts for AI systems tend to accumulate excessive privileges or persist long after their intended use. API keys may be shared informally among engineering teams, embedded directly in code, copied across repositories, or used in multiple environments beyond their intended scope.

The dynamic nature of AI development, characterized by rapid experimentation, short-lived training jobs, or ephemeral inference nodes, makes account life cycle management especially important. Weak identity practices can enable model misuse, unauthorized fine-tuning, extraction attacks, or uncontrolled consumption of third-party model APIs.

AI account management must therefore enforce strict separation of duties, avoid long-lived secrets, prevent privilege aggregation across stages of the AI life cycle, and ensure every identity tied to a model pipeline is traceable, auditable, and revocable.

Safeguards

CIS Control 5: Account Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
5.1	Establish and Maintain an Inventory of Accounts	Establish and maintain an inventory of all accounts managed in the enterprise. The inventory must at a minimum include user, administrator, and service accounts. The inventory, at a minimum, should contain the person's name, username, start/stop dates, and department. Validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.	●	●	●	Maintain an inventory of all user, administrator, and service accounts with access to AI models, training data, and inference pipelines for all model hosting and deployment types. Unmanaged accounts can be exploited to access sensitive models or poison training data. A complete inventory ensures only authorized access exists and facilitates swift revocation during offboarding.
5.2	Use Unique Passwords	Use unique passwords for all enterprise assets. Best practice implementation includes, at a minimum, an 8-character password for accounts using Multi-Factor Authentication (MFA) and a 14-character password for accounts not using MFA.	●	●	●	Where it is not possible to avoid password use through federation or similar means, enforce unique, strong passwords for all accounts accessing AI development platforms, model repositories, and SaaS AI services. Compromised credentials are a primary vector for model theft and data exfiltration. Unique passwords limit the blast radius if a single AI service or repository account is breached.
5.3	Disable Dormant Accounts	Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.	●	●	●	Remove dormant, stale, or unused identities in AI development, training, and model-management environments. Service accounts and API keys often remain active long after projects end. Regular cleanup prevents exploitation of forgotten credentials.

CIS Control 5: Account Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
5.4	Restrict Administrator Privileges to Dedicated Administrator Accounts	Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●	Restrict administrative access to AI infrastructure and model registries to dedicated accounts, separate from daily user or developer accounts. This prevents attackers who compromise a standard developer account from gaining full control over critical AI models and infrastructure, limiting the potential damage of a phishing attack.
5.5	Establish and Maintain an Inventory of Service Accounts	Establish and maintain an inventory of service accounts. The inventory, at a minimum, must contain department owner, review date, and purpose. Perform service account reviews to validate that all active accounts are authorized, on a recurring schedule at a minimum quarterly, or more frequently.		●	●	Inventory and track all service accounts used for automated model training, data pipelines, and API integrations. This should include all API keys, tokens, and other identity bearing assets that are used in non-human workflows. Service accounts and similar non-human identities often have high privileges and long lifespans; tracking them prevents unauthorized automation from accessing or exfiltrating sensitive data and helps identify anomalous machine behavior.
5.6	Centralize Account Management	Centralize account management through a directory or identity service.		●	●	Leverage federated identity and non-password authentication, such as Certificate Authorities (CAs) or OAuth with refresh tokens, for all LLM-related systems, applications, and services, especially SaaS-hosted LLMs. Centralization simplifies revocation and monitoring, reducing the risk of orphaned accounts retaining access to critical AI assets and providing a single point of control for security policies. Where passwordless methods cannot be implemented, passwords must expire on a defined schedule and be regularly rotated to maintain account security.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local inference clients often require API keys stored on endpoints; these must be governed (secure storage, rotation, revocation).
- Local experimentation should not use production service accounts or long-lived credentials.
- Per-user or per-workstation identities help prevent unauthorized lateral use.

Enterprise-Hosted Models

- GPU clusters, training pipelines, and inference systems frequently generate temporary workloads. Identities must be short-lived and automatically rotated.
- CI/CD pipelines for model deployment require scoped identities that cannot read training data or modify unrelated models.
- Fine-tuning environments must avoid privilege overlap with inference-serving environments.

SaaS-Hosted Models Accessed via API

- API keys, OAuth tokens, and service principals require tight scoping and frequent rotation.
- Separate identities for development, staging, and production prevent accidental leakage of sensitive data to external model providers during development.
- Logging must correlate SaaS API calls back to specific identities to ensure accountability for cost, data exposure, and behavior.

Additional AI LLM Considerations

- Explicitly tie identities to monitoring systems so anomalous model usage (extraction attempts, mass queries) can be attributed and contained.
- Identities that can fine-tune or retrain models must receive heightened scrutiny; misuse can alter system behavior or introduce backdoors.
- When using multiple models or model families, identities should be scoped per model to prevent accidental or unauthorized cross-model usage.
- If external plugins/tools are enabled (e.g., via model tool-calling interfaces), their identities must be governed under the same principles.
- Use distinct identities for each application, service, or integration that accesses LLM or model APIs. Shared API keys prevent tracing activity back to a specific system, user, or workflow. Unique identities enable revocation, monitoring, and access-scoping per use case.

Control 6: Access Control Management

Use processes and tools to create, assign, manage, and revoke access credentials and privileges for user, administrator, and service accounts for enterprise assets and software.

AI LLM Applicability

LLM systems introduce multiple layers of access surfaces that must be governed: model artifacts, training data, fine-tuning configurations, inference APIs, vector stores, application-level tools, and orchestration frameworks. Many AI systems rely on natural-language requests to trigger actions, but authorization must never be inferred from the text itself. Instead, identity, role, and context, rather than model interpretation, must determine permissions. Authorization decisions must be enforced outside the model; model output is never an authorization signal.

Access control for AI systems requires protecting:

- Model artifacts, including base models, fine-tuned versions, embeddings, and tokenizers, that may be sensitive IP or contain regulated data artifacts.
- Training/fine-tuning pipelines, which handle sensitive data and can be misused to alter model behavior.
- Inference endpoints, which may expose internal data or enable model extraction through excessive querying.
- Retrieval and augmentation stores (vector DBs, document sources), which often contain sensitive or proprietary materials.
- Tool invocation (e.g., code execution, browsing, search APIs), which may allow models to take real-world actions.

AI systems also often operate across multiple environments (e.g., development/testing/staging/production) and hosting types. Without strict access separation, identities may leak between environments, enabling unauthorized model modification, data access, or deployment of unapproved versions. Proper access control ensures that models behave predictably, that sensitive data is not mishandled, and that tools or capabilities cannot be triggered by malicious or accidental prompts.

Safeguards

CIS Control 6: Access Control Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
6.1	Establish an Access Granting Process	Establish and follow a documented process, preferably automated, for granting access to enterprise assets upon new hire or role change of a user.	●	●	●	Implement a formal, documented process for granting access to AI models, datasets, and computing resources based on business need and least privilege. Explicit approval workflows prevent unauthorized users or shadow teams from accessing sensitive model weights or training data, reducing the risk of IP theft and data leakage.
6.2	Establish an Access Revoking Process	Establish and follow a process, preferably automated, for revoking access to enterprise assets, through disabling accounts immediately upon termination, rights revocation, or role change of a user. Disabling accounts, instead of deleting accounts, may be necessary to preserve audit trails.	●	●	●	Establish a process to immediately revoke access to AI platforms, model repositories, and vector databases upon employee termination or role change. Prevents former employees from retaining access to proprietary models and sensitive training data, mitigating insider threats and unauthorized model exfiltration.
6.3	Require MFA for Externally-Exposed Applications	Require all externally-exposed enterprise or third-party applications to enforce MFA, where supported. Enforcing MFA through a directory service or SSO provider is a satisfactory implementation of this Safeguard.	●	●	●	Require MFA and SSO for all model management and deployment consoles, including SaaS portals and internal orchestration dashboards. Model management interfaces are high-value targets. Authentication must reflect the sensitivity of operations like fine-tuning, model replacement, or capability enablement.

CIS Control 6: Access Control Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
6.4	Require MFA for Remote Network Access	Require MFA for remote network access.	●	●	●	Enforce MFA for all remote connections to AI training clusters, model development environments, and inference servers. Protects high-value AI infrastructure from credential theft; attackers often target these environments to steal model IP or inject malicious code into training pipelines.
6.5	Require MFA for Administrative Access	Require MFA for all administrative access accounts, where supported, on all enterprise assets, whether managed on-site or through a service provider.	●	●	●	Require MFA for administrative access to model registries, cloud-based AI consoles, and centralized orchestration platforms. Compromise of an administrative AI account can lead to total system takeover, including the ability to poison models, delete datasets, or incur massive compute costs.
6.6	Establish and Maintain an Inventory of Authentication and Authorization Systems	Establish and maintain an inventory of the enterprise's authentication and authorization systems, including those hosted on-site or at a remote service provider. Review and update the inventory, at a minimum, annually, or more frequently.		●	●	Maintain an inventory of all identity providers and API gateway authentication mechanisms used for AI models and data stores. Ensures visibility into all entry points for AI resources, preventing "shadow" authentication methods that could be bypassed to gain unauthorized access to models or data.
6.7	Centralize Access Control	Centralize access control for all enterprise assets through a directory service or SSO provider, where supported.		●	●	Centralize access control decisions for AI resources through a unified policy engine or identity management system (e.g., SSO). This ensures consistent enforcement of security policies across diverse AI tools and platforms, preventing coverage gaps where access rules might be misconfigured locally.
6.8	Define and Maintain Role-Based Access Control	Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●	Enforce strict, role-based separation of duties for training data modification, model retraining, and production promotion. For example, this may introduce new role types including model publisher, model operator, prompt/policy maintainer, data curator, vector store admin, tool owner, application integrator, red team/evaluator. Unauthorized dataset changes or unapproved retraining jobs are the primary vectors for "data poisoning" and supply chain compromise. Training, fine-tuning, and deployment stages should utilize distinct identities (e.g., just-in-time roles) rather than shared credentials to limit blast radius. By tracking the full life cycle of identities accessing high-value artifacts, enterprises ensure that only authorized personnel can alter the model's fundamental behavior or promote new versions to production.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Endpoints running local LLMs may require local-only API keys or offline tokens; ensure they cannot access sensitive central resources.
- Local tools, such as document readers, or filesystem accessors, must be disabled or policy-restricted to prevent unauthorized data exposure.

Enterprise-Hosted Models

- Model stores, training clusters, vector DBs, and inference gateways must enforce identity-based access across all nodes and pipelines.
- Separate roles should govern training vs. inference vs. deployment, ensuring that no single identity has "full pipeline" access.
- Access control should integrate with Identity and Access Management (IAM) or zero-trust systems to enforce context-based policies across user, workload, and environment.

SaaS-Hosted Models Accessed via API

- API tokens should be tightly scoped:
 - One identity per application or service
 - No permissions for management operations unless explicitly needed
- Ensure that SaaS administrative consoles use enterprise Single Sign-On (SSO) and Multi-Factor Authentication (MFA).
- Validate whether the provider supports tenant-scoped roles or per-key restrictions, and select services that provide strong identity isolation.

Additional AI LLM Considerations

- Tools exposed to models (search, code execution, file access, browsing) must be governed through explicit IAM policy, not prompt-driven logic.
- Access to RAG sources or memory stores is as sensitive as access to internal knowledge bases; poisoning or unauthorized access can alter system behavior.
- Systems should flag or block prompts that appear to be privilege-escalation attempts (requesting tools or actions not allowed by policy).
- Developers must avoid creating “over-permissioned” AI integrations that grant broad or universal access to backend systems.
- If models invoke external APIs, ensure those APIs are governed with the same identity controls as human-driven systems.
- Base runtime authorization for tools, data retrieval, and inference APIs on identity policies, never model intent. LLMs must never serve as the arbiter of permission. If a model “decides” to access a tool or vector store based on a prompt, it can be manipulated via injection. Authorization must be enforced at the API or infrastructure level (hard-coded identity policies), ensuring that inference servers run with least-privilege identities and that access to sensitive augmentation sources is restricted regardless of what the model requests.

Control 7: Continuous Vulnerability Management

Develop a plan to continuously assess and track vulnerabilities on all enterprise assets within the enterprise’s infrastructure, in order to remediate, and minimize, the window of opportunity for attackers. Monitor public and private industry sources for new threat and vulnerability information.

AI LLM Applicability

LLM systems expand the vulnerability surface beyond traditional software stacks. These systems depend on:

- ML frameworks (e.g., PyTorch, TensorFlow)
- GPU drivers and CUDA/cuDNN libraries
- Model-serving runtimes (e.g., virtual large language model (vLLM), Triton, custom inference servers)
- Pre/post-processing pipelines
- Fine-tuning toolchains
- Tokenizers, embedding models, and vector databases
- SaaS-hosted model APIs and client SDKs

Many of these components update rapidly, and security defects can arise in unexpected areas, such as unsafe deserialization paths in model loaders, GPU kernel vulnerabilities, tokenizer logic bugs, or weaknesses in embedding pipelines.

Additionally, model-level vulnerabilities such as prompt-injection weaknesses, model extraction susceptibility, or documented jailbreak vectors function similarly to vulnerabilities in traditional applications. They may require configuration changes, updated guardrails, or controlled rollout of newer model versions.

Model behavior can change following fine-tuning, model replacement, framework updates, or driver patches. Continuous vulnerability management for AI systems must include observability of behavioral drift and post-update verification, since a technically “patched” environment could exhibit degraded or unsafe model responses.

Safeguards

CIS Control 7: Continuous Vulnerability Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
7.1	Establish and Maintain a Vulnerability Management Process	Establish and maintain a documented vulnerability management process for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Establish a process to continuously assess vulnerabilities in AI frameworks (e.g., PyTorch, TensorFlow), libraries, model dependencies, and SaaS services where LLMs are sourced. AI software stacks are complex and rapidly evolving; a formal process ensures that new vulnerabilities are identified and addressed systematically before they can be exploited.
7.2	Establish and Maintain a Remediation Process	Establish and maintain a risk-based remediation strategy documented in a remediation process, with monthly, or more frequent, reviews.	●	●	●	Maintain a patch and update strategy for AI runtimes, serving frameworks, GPU drivers, and related dependencies. Patching must account for compatibility testing, model behavior evaluation, and risk-based prioritization to avoid introducing instability while resolving vulnerabilities.

CIS Control 7: Continuous Vulnerability Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
7.3	Perform Automated Operating System Patch Management	Perform operating system updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●	Apply OS patching to all AI compute and control-plane hosts, including GPU drivers/kernel updates with staged rollout and rollback.
7.4	Perform Automated Application Patch Management	Perform application updates on enterprise assets through automated patch management on a monthly, or more frequent, basis.	●	●	●	Track updates to AI frameworks, drivers, and model-serving runtimes through standard patch management processes (e.g., CUDA/cuDNN, model serving runtimes, container images, SDKs, vector databases, notebook platforms). AI software evolves quickly, and updates may fix vulnerabilities or compatibility issues. Visibility ensures alignment with patch and configuration baselines.
7.5	Perform Automated Vulnerability Scans of Internal Enterprise Assets	Perform automated vulnerability scans of internal enterprise assets on a quarterly, or more frequent, basis. Conduct both authenticated and unauthenticated scans.		●	●	Continuously scan AI-hosting environments (containers, VMs, serverless functions) for outdated or unpatched components, including ML-specific dependencies. AI workloads frequently rely on custom containers or pinned library versions; ensuring that these images remain up to date requires deliberate scanning and tracking.
7.6	Perform Automated Vulnerability Scans of Externally-Exposed Enterprise Assets	Perform automated vulnerability scans of externally-exposed enterprise assets. Perform scans on a monthly, or more frequent, basis.		●	●	Automatically scan externally exposed AI API endpoints and web interfaces for vulnerabilities on a regular basis. Public-facing AI services are constant targets; scanning identifies weaknesses that could be exploited to access models or data before attackers can find them.
7.7	Remediate Detected Vulnerabilities	Remediate detected vulnerabilities in software through processes and tooling on a monthly, or more frequent, basis, based on the remediation process.		●	●	Remediate identified vulnerabilities in AI software and infrastructure in a timely manner based on risk severity. Prompt remediation closes the window of opportunity for attackers to exploit known flaws in the AI environment, maintaining the security and integrity of AI systems.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local model runtimes, Python environments, and GPU drivers require scanning and patching through endpoint vulnerability tools.
- Notebook environments may install Machine Learning (ML) libraries dynamically; these must be tracked and scanned for vulnerable versions.
- Local inference clients should be monitored for outdated or unsupported model files.

Enterprise-Hosted Models

- Containers and images used for training or inference must be part of automated vulnerability scanning pipelines.
- GPU clusters require coordinated patching, as CUDA-level updates may break compatibility with ML frameworks; therefore, behavioral regression tests should accompany such updates.
- Model registries should integrate with vulnerability feeds to identify issues in model artifacts or dependencies.

SaaS-Hosted Models Accessed via API

- Enterprises must monitor provider advisories, release notes, or model version changes that may introduce new weaknesses or modify model behavior.
- Client SDKs used internally must be patched promptly when providers release updates.
- Access patterns should be monitored for signs of exploitation (e.g., abnormal heavy querying that suggests model extraction attempts).

Additional AI LLM Considerations

- Tokenizers and embedding models may have their own vulnerability disclosures; mismatches across the ecosystem can cause failure modes or security flaws.
- Fine-tuning pipelines may include custom script execution, creating pathways for code-injection vulnerabilities if dependencies are not managed and scanned.
- Some models are distributed with unsafe default configuration (e.g., unrestricted plugin loading, debugging endpoints); securing these defaults is part of vulnerability management.
- A model's vulnerability profile evolves as new attack techniques are discovered; LLM-specific vulnerability intelligence must be tracked alongside traditional CVEs.
- Enterprises should assess the impact of model upgrades, as small changes in content-filtering or reasoning behavior may create new security concerns.

Control 8: Audit Log Management

Collect, alert, review, and retain audit logs of events that could help detect, understand, or recover from an attack.

AI LLM Applicability

AI systems generate unique logs that differ significantly from traditional application logs. LLM-related logs may include:

- Prompts, completions, embeddings, and their associated metadata, which often contain sensitive information.
- Model API usage statistics, including rate-limiting events and anomalous query patterns.
- Training and fine-tuning events, such as dataset versions, hyperparameters, and approvals.
- Access to augmentation sources (e.g., vector databases, document stores), which may include sensitive enterprise knowledge.
- Tool invocation activity, when models are integrated with external tools or APIs.

Logging is essential for detecting:

- Data leakage
- Prompt-injection attempts
- Model extraction activity
- Abnormal or abusive traffic
- Unauthorized model usage
- Unexpected model behavior changes
- Poisoning attempts in RAG or memory systems

Since LLM logs often contain sensitive data, enterprises must classify and protect them with the same rigor as the underlying data sources. SaaS-hosted models introduce additional complexity: enterprises must retrieve, store, and integrate provider logs, and ensure log retention aligns with compliance requirements.

Safeguards

CIS Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
8.1	Establish and Maintain an Audit Log Management Process	Establish and maintain a documented audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Define and maintain a process for collecting and reviewing logs from LLMs, APIs, and training environments. Ensures that necessary evidence is available to detect, investigate, and respond to security incidents involving AI systems, enabling effective accountability and forensic analysis.
8.2	Collect Audit Logs	Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●	Log all inference traffic, life cycle events, data access, and multi-modal inputs with granular metadata. Comprehensive logging is the backbone of AI forensics. API logs with strict identity attribution reveal abuse patterns, while recording training and fine-tuning events ensures accountability for model evolution and behavioral drift. Crucially, monitoring access to underlying assets, such as training datasets and vector stores, is the only way to detect data poisoning or exfiltration attempts early. Furthermore, as models expand into multi-modal capabilities, logging strategies must adapt to capture images and audio (via hashing or metadata) without violating privacy rules, recognizing that these opaque artifacts can bypass standard text-based inspection tools.

CIS Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
8.3	Ensure Adequate Audit Log Storage	Ensure that logging destinations maintain adequate storage to comply with the enterprise's audit log management process.	●	●	●	Provision sufficient storage to retain logs from LLM systems, including high-volume inference logs, for the required retention period. Prevents the loss of critical forensic data due to storage exhaustion, ensuring logs are available for post-incident analysis even for high-throughput AI applications.
8.4	Standardize Time Synchronization	Standardize time synchronization. Configure at least two synchronized time sources across enterprise assets, where supported.		●	●	Synchronize clocks across all AI infrastructure components to ensure accurate correlation of events in logs. Accurate timestamps are essential for reconstructing the timeline of an attack that spans multiple AI systems and services, enabling effective root cause analysis.
8.5	Collect Detailed Audit Logs	Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.		●	●	Configure LLM systems to generate detailed logs, including user inputs, model outputs (where appropriate), and system state changes. Detailed logs provide the context needed to understand the nature and impact of complex attacks against AI models, such as prompt injection or data extraction.
8.6	Collect DNS Query Audit Logs	Collect DNS query audit logs on enterprise assets, where appropriate and supported.		●	●	Collect DNS logs for AI-related domains (e.g., model providers, model hubs, artifact registries) and alert on unknown AI endpoints.
8.7	Collect URL Request Audit Logs	Collect URL request audit logs on enterprise assets, where appropriate and supported.		●	●	Log and attribute outbound HTTP/API requests initiated by models or agents, specifically distinguishing "tool use" traffic from standard infrastructure updates. Models used in conjunction with browsing or URL fetching tool capabilities can be manipulated (e.g., via a Server-Side Request Forgery (SSRF) vulnerability) to access internal metadata services or exfiltrate data to unauthorized external APIs. Collecting the prompt response data from the LLM about what URLs or information from URLs the LLM is requesting tools to fetch completes the picture needed for IR and other investigations of suspicious activity.
8.8	Collect Command-Line Audit Logs	Collect command-line audit logs. Example implementations include collecting audit logs from PowerShell®, BASH™, and remote administrative terminals.		●	●	Capture execution logs from "code interpreter" sandboxes and training environments, ensuring transient containers offload logs to central storage before termination. AI systems capable of generating and executing code (e.g., Python agents) act as remote shells; without logs, malicious code execution is untraceable.
8.9	Centralize Audit Logs	Centralize, to the extent possible, audit log collection and retention across enterprise assets in accordance with the documented audit log management process. Example implementations primarily include leveraging a SIEM tool to centralize multiple log sources.		●	●	Aggregate logs from all LLM components into a central repository for unified analysis and correlation. Centralization enables the detection of complex attack patterns that span multiple systems and provides a single source of truth for investigations across the AI ecosystem.
8.10	Retain Audit Logs	Retain audit logs across enterprise assets for a minimum of 90 days.		●	●	Retain AI-related audit logs for a defined period to support long-term trend analysis and forensic investigations. Ensures that historical data is available to investigate incidents that may not be detected immediately, supporting compliance and long-term security monitoring.
8.11	Conduct Audit Log Reviews	Conduct reviews of audit logs to detect anomalies or abnormal events that could indicate a potential threat. Conduct reviews on a weekly, or more frequent, basis.		●	●	Review logs for adversarial interaction patterns, excessive errors, and unexpected behavioral drift. Behavioral signals, such as rapid sequences of adversarial prompts or excessive error rates, often serve as the earliest indicators of an active exploitation attempt, model extraction campaign, or automated probing for jailbreaks. Simultaneously, "silent" failures manifested as behavioral drift, where model outputs degrade or bypass safety filters following updates, fine-tuning, or configuration changes can reveal unintentional side effects that introduce critical vulnerabilities. Treating these anomalies as security signals requires rigorous investigation to distinguish between benign operational noise and malicious tradecraft, ensuring that both active attacks and passive degradation of safety posture are identified and remediated before they result in data leakage or service compromise.

CIS Control 8: Audit Log Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
8.12	Collect Service Provider Logs	Collect service provider logs, where supported. Example implementations include collecting authentication and authorization events, data creation and disposal events, and user management events.				Integrate SaaS model API telemetry (if available) with network monitoring to detect suspicious or abnormal usage patterns. Provider-side logs often reflect rate limits, warnings, or policy issues. Correlating this data with network telemetry improves detection of misuse or compromise.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local clients may store logs on disk. Ensure local logging is encrypted and routed to central SIEM when required.
- Prompt logs from local experimentation may contain sensitive data; retention must be governed centrally.
- Developers often use notebooks that save interaction history automatically; these must be monitored and restricted.

Enterprise-Hosted Models

- Inference servers, RAG components, and vector stores must emit logs to central log aggregation systems.
- Training environments need detailed auditing for dataset access and model changes.
- Multi-node training and distributed inference should use consistent log schemas for correlation.

SaaS-Hosted Models Accessed via API

- Providers may offer partial logs (e.g., token usage, errors, rate limits) but not full prompt/completion logs. Enterprises must understand what is and is not available.
- Leverage existing auditing tools (e.g., CASB, EDR) to get a more robust picture of the activities associated with LLM activities.
- Logs that cannot be retrieved (e.g., provider-retained data) must be covered by contractual controls.
- Client-side logs must supplement gaps in provider visibility.

Additional AI LLM Considerations

- Monitor model outputs for high-similarity repetition or anomalous entropy patterns indicative of extraction or training data leakage.
- Logs themselves are sensitive and must be classified accordingly, as prompt logs often contain PII, proprietary content, or regulated data.
- Embedding generation logs may expose sensitive semantic information even without raw text.
- Log volume may be extremely high for LLM-intensive workflows, and enterprises must plan for storage, retention, and query scalability.
- Red team and evaluation activity may need explicit labeling so that it is not mistaken for adversarial behavior.
- If models invoke tools, tool-level logging must correlate tool usage with the originating model interaction.

Control 9: Email and Web Browser Protections

Improve protections and detections of threats from email and web vectors, as these are opportunities for attackers to manipulate human behavior through direct engagement.

AI LLM Applicability

In LLM systems, email and browsers serve as both human interfaces and data ingress and egress paths. Users may paste sensitive content into web-based AI chat interfaces, upload documents or images to AI-powered web tools, or use browser extensions that silently send page content to external AI services. At the same time, AI tools may perform browsing or send emails on behalf of users, effectively acting as automated web and email clients.

If these channels are not governed, they can create uncontrolled pathways that bypass established controls, including:

- Users exposing sensitive or regulated data to unapproved public AI services.
- Browser extensions exfiltrating web content or credentials to external LLMs.
- AI tools browsing the web or sending emails without passing through corporate secure web gateways (SWG) or email security platforms.
- AI tools reading emails or other messages on behalf of users and are subject to prompt injection and other attacks that can be embedded in the messages.
- File uploads to AI interfaces skipping malware scanning and DLP inspection.

Email and browser protections for AI need to ensure that AI-mediated web and email traffic is subject to the same or stricter controls as human traffic. The goal is not to treat AI channels as special exceptions, but to integrate them into existing secure web and email architectures, preserving visibility, policy enforcement, and data protection.

Safeguards

CIS Control 9: Email and Web Browser Protections

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
9.1	Ensure Use of Only Fully Supported Browsers and Email Clients	Ensure only fully supported browsers and email clients are allowed to execute in the enterprise, only using the latest version of browsers and email clients provided through the vendor.				Require supported browsers and managed configurations for any AI web user interface usage and any AI extensions. AI extensions for web browsers or email clients should generally be prohibited due to the risk of prompt injection and other common vulnerabilities introduced by third-party model integrations.
9.2	Use DNS Filtering Services	Use DNS filtering services on all end-user devices, including remote and on-premises assets, to block access to known malicious domains.				DNS filtering must block unapproved AI services and known malicious domains used for AI credential theft and tool poisoning.

CIS Control 9: Email and Web Browser Protections

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
9.3	Maintain and Enforce Network-Based URL Filters	Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.		●	●	Enforce strict URL filtering on agent egress traffic and block access to unapproved external AI services. AI agents functioning as autonomous web clients must operate behind the same Secure Web Gateway (SWG) and DLP controls as human users. Without strict egress filtering, a compromised agent could be manipulated via prompt injection to exfiltrate data to arbitrary URLs or communicate with command-and-control servers, bypassing standard browser protections. Furthermore, network-based filtering is the primary defense against "Shadow AI." By treating external generative AI services as a restricted category, enterprises ensure that neither internal agents nor human employees can transmit sensitive data to unvetted public models, enforcing a "default-deny" posture for all unauthorized inference endpoints.
9.4	Restrict Unnecessary or Unauthorized Browser and Email Client Extensions	Restrict, either through uninstalling or disabling, any unauthorized or unnecessary browser or email client plugins, extensions, and add-on applications.		●	●	Monitor and control browser extensions or plug-ins that send data to external AI services. Extensions can silently capture page content, forms, and credentials and forward them to AI providers. Extension management, allowlists, and periodic review are critical to prevent unapproved or malicious add-ons from exfiltrating data.
9.5	Implement DMARC	To lower the chance of spoofed or modified emails from valid domains, implement DMARC policy and verification, starting with implementing the Sender Policy Framework (SPF) and the DomainKeys Identified Mail (DKIM) standards.		●	●	While DMARC is not directly applicable to LLMs, DMARC reduces spoofing used to trick users into granting AI access, installing AI plugins, or sending sensitive data to fake AI portals and should be considered a protective layer to ensure LLM safety.
9.6	Block Unnecessary File Types	Block unnecessary file types attempting to enter the enterprise's email gateway.		●	●	No Additional AI LLM Guidance
9.7	Deploy and Maintain Email Server Anti-Malware Protections	Deploy and maintain email server anti-malware protections, such as attachment scanning and/or sandboxing.			●	No Additional AI LLM Guidance

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local LLM tools embedded into browsers (e.g., sidebars, extensions) may route data locally or to external APIs. Endpoint management should ensure those tools are either approved and configured or removed.
- Workstations used for AI-assisted browsing must still respect corporate SWG and DLP policies. Local tools must not configure direct, unmanaged connections to external AI endpoints.

Enterprise-Hosted Models

- AI-powered internal web applications (e.g., "talk to your data" portals) must integrate with existing web security controls for upload scanning and outbound filtering.
- When enterprise-hosted agents perform web browsing, traffic should originate from tightly controlled egress points or service networks, preserving URL filtering and logging.
- Internal AI portals should clearly indicate their approved status and provide guidance to users on what types of data are allowed.

SaaS-Hosted Models Accessed via API

- Access to SaaS AI web UIs should be mediated through SWGs where feasible, enabling domain-based allowlists and content controls.
- Some SaaS providers expose both web UIs and APIs. Enterprises must ensure both are governed consistently, not allowing the web UI to be used as a “backdoor” around stricter API policies.
- For SaaS tools that can send emails or access external URLs, verify how those capabilities are controlled and logged, and whether they respect corporate domains and policies.

Additional AI LLM Considerations

- AI summarization or drafting of emails does not eliminate phishing risk; if AI tools send emails, they may be used to scale social engineering attempts. Outbound AI-generated emails must be governed by the same, or tighter, policies as human-generated email.
- AI systems that browse the web may follow links in untrusted content, potentially accessing malicious sites. URL filtering and sandboxing are just as important for AI bots as for human users.
- If models can accept screenshots or copy/paste from web pages, data from secure applications (e.g., internal portals) could be exfiltrated to external AI services. DLP and user training must recognize this pattern.
- AI browser extensions and AI-powered browsers marketed as “productivity helpers” can be high-risk. Enterprises should block them by default and only allow vetted, enterprise-integrated extensions.
- Logging and monitoring should distinguish AI-driven web activity from human-driven browsing to support investigation and policy tuning.

Control 10: Malware Defenses

Prevent or control the installation, spread, and execution of malicious applications, code, or scripts on enterprise assets.

AI LLM Applicability

AI systems introduce new malware risks in several ways. Model-hosting environments (both local and server-based) typically run complex ML frameworks, Python environments, GPU drivers, and containerized stacks. These components frequently include native code, custom kernels, or dynamic loading behavior, expanding opportunities for exploitation.

LLM- generated code, scripts, or configurations can introduce vulnerabilities or malware if executed without review. Even when the intent is benign (e.g., “help me write this script”), generated code may contain insecure patterns, backdoor-like logic, or harmful payloads. AI systems integrated with tools capable of file creation, code execution, browsing, or package installation may accidentally or deliberately interact with malicious content in ways that bypass traditional human review.

Endpoints that run local LLMs or notebooks often disable protections for performance or convenience, or they may store model files and caches that could be tampered with by attackers to introduce malicious behavior.

For these reasons, LLM-related environments must be treated as high-risk compute surfaces requiring strong malware protections, integrity checks, and guardrails around model-generated code.

Safeguards

CIS Control 10: Malware Defenses

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
10.1	Deploy and Maintain Anti-Malware Software	Deploy and maintain anti-malware software on all enterprise assets.	●	●	●	Deploy and maintain anti-malware software on all servers and workstations used for AI development and inference. This protects the underlying infrastructure from malware infections that could compromise AI models and sensitive data, ensuring the availability and integrity of AI services.
10.2	Configure Automatic Anti-Malware Signature Updates	Configure automatic updates for anti-malware signature files on all enterprise assets.	●	●	●	Ensure anti-malware signatures are automatically updated on AI infrastructure to detect the latest threats. Rapid updates are essential to defend against newly emerging malware strains that could target AI systems, minimizing the window of vulnerability.
10.3	Disable Autorun and Autoplay for Removable Media	Disable autorun and autoplay auto-execute functionality for removable media.	●	●	●	No Additional AI LLM Guidance
10.4	Configure Automatic Anti-Malware Scanning of Removable Media	Configure anti-malware software to automatically scan removable media.		●	●	No Additional AI LLM Guidance
10.5	Enable Anti-Exploitation Features	Enable anti-exploitation features on enterprise assets and software, where possible, such as Microsoft® Data Execution Prevention (DEP), Windows® Defender Exploit Guard (WDEG), or Apple® System Integrity Protection (SIP) and Gatekeeper™.		●	●	No Additional AI LLM Guidance
10.6	Centrally Manage Anti-Malware Software	Centrally manage anti-malware software.		●	●	Prevent local LLM runtimes, notebooks, or developer tools from bypassing host malware defenses. Developers may disable scanning to improve performance or convenience. However, ML environments often install numerous packages that could contain undetected malicious components if not adequately scanned.

CIS Control 10: Malware Defenses

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
10.7	Use Behavior-Based Anti-Malware Software	Use behavior-based anti-malware software.		●	●	Apply upload scanning, malware detection, and content inspection to file and image uploads in AI web interfaces. When users upload documents or images to AI tools (internal or external), those uploads must be scanned for malware and inspected for sensitive data according to existing web upload policies.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local LLM packages (e.g., model weights, libraries, tokenizer files) should be scanned during installation and monitored for changes.
- Notebook environments often install third-party packages dynamically. These must be governed by EDR and package scanning policies.
- Developers may run models in “performance mode” with relaxed security. Endpoint policies must prevent such bypasses.

Enterprise-Hosted Models

- Containers or VMs used for inference and fine-tuning must be scanned both at build time and at runtime.
- GPU servers, often running specialized drivers or custom kernels, must be part of the enterprise’s vulnerability and malware monitoring strategy.
- Multi-tenant training clusters require strict file-system isolation and malware scanning to prevent contamination between workloads.

SaaS-Hosted Models Accessed via API

- While the provider handles infrastructure malware defenses, client-side environments that interact with SaaS models still require scanning to prevent malicious inputs or outputs from entering the enterprise environment.
- Logs, files, and code downloaded from SaaS model workflows must be validated before use.
- Enterprises should confirm provider assurances regarding malware scanning of uploaded content (documents, images, or files submitted for analysis).

Additional AI LLM Considerations

- Code-generation use cases (e.g., DevOps, scripting, automation) significantly increase malware risk. Enterprises must ensure validation pipelines catch unsafe or malicious logic and hold a high bar for any “auto run” features.
- LLMs can be manipulated into producing obfuscated or harmful code; monitoring tools should detect repeated attempts to generate exploit patterns.
- Scan ingested documents for non-visual or hidden text layers (e.g., white-on-white text, metadata injection, prompts embedded in URLs) prior to embedding/indexing.
- Retrieval pipelines may ingest malicious content (e.g., HTML with embedded scripts); ingestion should use sanitization and scanning before indexing.
- Some model-serving frameworks dynamically load CUDA kernels or low-level operations. Tampering with these files could compromise servers.
- Tools that allow execution (e.g., “code interpreters”) must use hardened sandboxes with strict isolation and egress restrictions.

Control 11: Data Recovery

Establish and maintain data recovery practices sufficient to restore in-scope enterprise assets to a pre-incident and trusted state.

AI LLM Applicability

LLM systems introduce new categories of high-value data that must be backed up and recoverable. This includes:

- Training datasets, which often represent months or years of curation effort
- Fine-tuning corpora, including sensitive or proprietary customer/context data
- Model artifacts, such as checkpoints, tokenizer files, and inference-ready versions
- Embeddings and vector stores, which may represent scaled semantic indexes for enterprise content
- Evaluation data, model configuration files, and related metadata
- Long-term or persistent memory, if implemented
- RAG document stores, indexes, and preprocessing pipelines

Losing any of these can disrupt operations or impair the enterprise's ability to validate model behavior, revert unsafe changes, or maintain compliance. Training and fine-tuning pipelines also depend on reproducibility. Recoverable datasets and model artifacts allow security teams to investigate incidents, compare model behavior across versions, and re-establish safe baselines after a compromise or model poisoning event.

Since model behaviors can change subtly due to corruption, poisoning, or unintended modification, data recovery for AI systems must ensure both availability and integrity, including verification that restored datasets and model artifacts have not been tampered with. Recovery procedures should extend to model-serving stacks as well, since inference runtimes, environment configurations, and orchestration logic are critical to restoring operational AI workflows.

Safeguards

CIS Control 11: Data Recovery

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
11.1	Establish and Maintain a Data Recovery Process	Establish and maintain a documented data recovery process that includes detailed backup procedures. In the process, address the scope of data recovery activities, recovery prioritization, and the security of backup data. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Establish a process for the backup and recovery of AI models, training data, and configuration files. Ensures business continuity by enabling the restoration of critical AI capabilities following data loss or corruption events, minimizing downtime and operational impact.
11.2	Perform Automated Backups	Perform automated backups of in-scope enterprise assets. Run backups weekly, or more frequently, based on the sensitivity of the data.	●	●	●	Back up model registry states, prompt/policy configurations, evaluation datasets, RAG corpora, and the CI/CD artifacts (including embedding pipelines) required to rebuild or verify AI-enabled systems. These assets represent core intellectual property and must be recoverable for disaster recovery, compliance, and incident response. Backups must reflect appropriate sensitivity and retention requirements. Version and back up model artifacts, including checkpoints, tokenizer files, configuration metadata, and evaluation results. Model artifacts allow enterprises to revert to trusted versions and validate changes. Backup processes should ensure that all elements needed to reproduce a model's behavior are preserved.

CIS Control 11: Data Recovery

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
11.3	Protect Recovery Data	Protect recovery data with equivalent controls to the original data. Reference encryption or data separation, based on requirements.	●	●	●	Encrypt and restrict access to backups of AI models and data to prevent unauthorized access or tampering. Backup data contains the same sensitive IP and information as production systems and requires equivalent protection to prevent data breaches or integrity loss.
11.4	Establish and Maintain an Isolated Instance of Recovery Data	Establish and maintain an isolated instance of recovery data. Example implementations include, version controlling backup destinations through offline, cloud, or off-site systems or services.	●	●	●	Maintain an isolated (offline or immutable) copy of AI backups to protect against ransomware and destructive attacks. This ensures that a clean, uncorrupted copy of data is available for recovery even if the primary network is compromised or data is encrypted by attackers.
11.5	Test Data Recovery	Test backup recovery quarterly, or more frequently, for a sampling of in-scope enterprise assets.		●	●	Test recovery procedures for all AI components, verifying the integrity of restored artifacts and maintaining rollback capabilities. Restoring a corrupted or poisoned model checkpoint can reintroduce vulnerabilities or unsafe behaviors. Therefore, recovery tests must go beyond simple file availability to include cryptographic integrity verification and behavioral validation of restored datasets, embeddings, and weights. Furthermore, effective resilience requires granular rollback capabilities, allowing teams to revert not just the model, but also its specific configuration, training data, and stored memory states, ensuring that the system can be returned to a precise "known-good" baseline following contamination or an adversarial compromise.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local model files, cached embeddings, and user-specific fine-tuning artifacts may not be stored centrally. Policies should ensure recoverability or prevent local-only critical data.
- Restoration for endpoints may focus on re-provisioning models from approved central sources rather than backing up local artifacts.

Enterprise-Hosted Models

- Training clusters and inference servers rely on consistent access to model files, datasets, and vector stores. Backups must include both data and orchestration/configuration (e.g., container manifests, training scripts).
- Vector databases can be large and continuously updated; backup strategies may require incremental snapshots or log-based replication.
- Model registries must be included in backups to ensure consistent availability of approved versions.

SaaS-Hosted Models Accessed via API

- SaaS providers generally maintain availability and recovery for their platforms, but enterprise-owned data, such as logs, prompts, datasets used for fine-tuning, or retrieval corpora stored outside provider infrastructure, must still be backed up by the enterprise.
- Enterprises should confirm whether the provider retains fine-tuned model snapshots or allows export for independent backup.
- Local SDK configs, integration code, and policy layers must be part of enterprise backups even when models are provider-managed.

Additional AI LLM Considerations

- Model poisoning incidents often require rolling back to known-good versions; recovery plans must include secure rollback paths.
- Embeddings and vector databases may require special handling due to their size and rate of change. Corrupted embeddings can degrade entire retrieval pipelines.
- Training pipelines include transient intermediate datasets; enterprises must decide which parts must be preserved versus regenerated.
- Fine-tuning jobs may incorporate user-generated data. Retention policies must ensure compliance with data classification and deletion requirements.
- Recovery must consider not just data availability but also behavioral fidelity to ensure that a restored model performs and behaves like the original.

Control 12: Network Infrastructure Management

Establish, implement, and actively manage (track, report, correct) network devices, in order to prevent attackers from exploiting vulnerable network services and access points.

AI LLM Applicability

LLM systems often rely on specialized network paths, including high-bandwidth links for training clusters, secure endpoints for inference APIs, and connectivity between model-serving systems, vector stores, and other application components. AI workloads frequently cross boundaries, spanning across multiple environments and endpoints, making network segmentation and controlled connectivity essential to limiting the blast radius and preventing unauthorized data flows.

Common network risks in AI environments include:

- Training clusters with broad internal access that can be abused if not properly segmented.
- Inference servers exposed to public networks or open IP ranges without authentication.
- RAG pipelines that rely on external retrieval sources or internal document repositories accessible from multiple zones. Note: While RAG is not in scope for this guide, we note the risk for completeness.
- Developer or research networks that unintentionally have paths to production model endpoints or sensitive data stores.
- Tools invoked by LLMs (e.g., browsers, search APIs, code execution endpoints) that may initiate outbound traffic.

Incorrect network configuration may allow adversaries to intercept model traffic, access training data, compromise inference nodes, or route model-generated requests outside approved boundaries. Network controls must therefore encompass the full AI life cycle, including training, fine-tuning, inference, augmentation, orchestration, and tool use.

Safeguards

CIS Control 12: Network Infrastructure Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
12.1	Ensure Network Infrastructure is Up-to-Date	Ensure network infrastructure is kept up-to-date. Example implementations include running the latest stable release of software and/or using currently supported network as a service (NaaS) offerings. Review software versions monthly, or more frequently, to verify software support.	●	●	●	No Additional AI LLM Guidance
12.2	Establish and Maintain a Secure Network Architecture	Design and maintain a secure network architecture. A secure network architecture must address segmentation, least privilege, and availability, at a minimum. Example implementations may include documentation, policy, and design components.		●	●	Isolate AI assets behind strict network boundaries, enforcing least-privilege access and allowlisted egress. Vector stores and model artifacts often contain the enterprise's most sensitive intellectual property; they must be isolated from general networks to prevent unauthorized discovery. Enforcing least-privilege access at the network layer limits the blast radius, ensuring that only specific, authorized systems can reach these high-value targets. Crucially, restricting outbound connectivity is the primary defense against autonomous data exfiltration; without strict egress filtering to approved domains, a compromised or hallucinating model could send proprietary outputs to external endpoints or retrieve malicious content that fundamentally alters its safety alignment.

CIS Control 12: Network Infrastructure Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
12.3	Securely Manage Network Infrastructure	Securely manage network infrastructure. Example implementations include version-controlled Infrastructure-as-Code (IaC), and the use of secure network protocols, such as SSH and HTTPS.		●	●	Use authenticated gateways/API proxies (or mutual TLS) so that only authorized systems can reach model endpoints. Inference APIs should not be directly exposed to user networks; gateways provide logging, identity enforcement, rate limiting, and consistent policy application.
12.4	Establish and Maintain Architecture Diagram(s)	Establish and maintain architecture diagram(s) and/or other network system documentation. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Maintain up-to-date diagrams of LLM system network architecture, including data flows between models, data stores, and external APIs. Accurate diagrams are essential for understanding the attack surface, planning defenses, and conducting incident response in complex AI environments.
12.5	Centralize Network Authentication, Authorization, and Auditing (AAA)	Centralize network AAA.		●	●	Centralize network authorization to bind identity to permitted network paths for AI workloads. Define and centrally enforce network allowlists per user, administrator, or service account to dictate exactly where an identity can authenticate from and which specific AI services or model APIs it is authorized to call, preventing unintended or unauthorized cross-platform access.
12.6	Use of Secure Network Management and Communication Protocols	Adopt secure network management protocols (e.g., 802.1X) and secure communication protocols (e.g., Wi-Fi Protected Access 2 (WPA2) Enterprise or more secure alternatives).		●	●	No Additional AI LLM Guidance
12.7	Ensure Remote Devices Utilize a VPN and are Connecting to an Enterprise's AAA Infrastructure	Require users to authenticate to enterprise-managed VPN and authentication services prior to accessing enterprise resources on end-user devices.		●	●	No Additional AI LLM Guidance
12.8	Establish and Maintain Dedicated Computing Resources for All Administrative Work	Establish and maintain dedicated computing resources, either physically or logically separated, for all administrative tasks or tasks requiring administrative access. The computing resources should be segmented from the enterprise's primary network and not be allowed internet access.			●	Use dedicated, hardened workstations for administrative tasks on AI clusters and sensitive model infrastructure. Reduces the risk of admin credentials being stolen by malware on a general-purpose workstation and used to compromise critical AI systems.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local models may attempt to fetch updates or make outbound API calls; outbound traffic must be routed through standard enterprise security controls.
- Endpoints running LLM-based tools should not be allowed to directly access internal model stores unless explicitly authorized.
- When local tools invoke browsing or search actions, DNS and egress controls must ensure they cannot reach unapproved domains.

Enterprise-Hosted Models

- Training clusters often reside in isolated high-performance networks. Segmentation must ensure they cannot reach production APIs or sensitive stores unless required.
- Inference servers should sit behind controlled ingress points (e.g., reverse proxies, API gateways) rather than have direct IP exposure.
- Vector stores and RAG pipelines should be placed in restricted network segments, with only the specific inference components allowed access.

SaaS-Hosted Models Accessed via API

- When calling SaaS model APIs, outbound connections should originate from controlled egress networks to ensure that SWG, TLS inspection, DLP, and logging apply.
- If SaaS services require inbound connections (rare), traffic must terminate at secure gateways that authenticate and log all activity.
- Multi-cloud routing policies must ensure that model traffic does not inadvertently traverse untrusted regions or uncontrolled service paths.

Additional AI LLM Considerations

- LLM tools that invoke external HTTP requests must be restricted so that generated prompts cannot direct traffic to arbitrary destinations.
- DNS filtering must treat AI tools the same as human users; AI-generated traffic must not bypass content filtering or threat categorization.
- Model extraction and distillation attacks often depend on sustained high-volume traffic or coordinated patterns distributed across multiple accounts. Network throttling and rate limiting through gateways are essential but should be supplemented by behavioral fingerprinting to detect anomalous “hydra” account clusters and query patterns.
- Implement heightened identity verification for access pathways commonly exploited for model theft, such as educational, startup, or security research programs, to prevent the creation of fraudulent accounts used for industrial-scale distillation.
- Networking for distributed training (multi-node, multi-GPU) may require high-throughput protocols that bypass standard firewalls; compensating controls must be in place.
- Internal AI research environments must be isolated to prevent unauthorized access to production data or model artifacts.

Control 13: Network Monitoring and Defense

Operate processes and tooling to establish and maintain comprehensive network monitoring and defense against security threats across the enterprise’s network infrastructure and user base.

AI LLM Applicability

AI systems generate distinct and sometimes high-volume network patterns that must be monitored carefully. Inference servers, RAG pipelines, vector stores, training clusters, and model-management systems often communicate across specialized network paths. These systems may interact with both internal resources (e.g., datasets, document stores, logging infrastructure) and external endpoints (e.g., SaaS models, API gateways, third-party services).

Attackers may attempt to exploit these network channels to:

- Extract model parameters or proprietary knowledge via high-volume queries
- Send malicious or adversarial content to training or inference endpoints
- Access augmentation/RAG stores for reconnaissance or poisoning
- Interfere with distributed training or model-serving infrastructure
- Manipulate inference behavior by injecting unexpected network inputs
- Direct AI tools or agents toward malicious domains or payloads

Traditional network monitoring tools may not automatically recognize model-level extraction attempts, abnormal inference traffic, or misuse of model endpoints. Monitoring must therefore evolve to recognize AI-specific behaviors such as anomalous prompt frequency, irregular result sizes, atypical access patterns to vector stores, or unexpected egress to unapproved AI APIs.

Additionally, multimodal or tool-enabled systems may interact with external HTTP endpoints, raising the need for deeper inspection of AI-driven outbound traffic. Network defenses should ensure that AI systems, especially those capable of browsing or interacting with external resources, operate entirely within the enterprise’s monitored and policy-controlled network boundaries.

Safeguards

CIS Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
13.1	Centralize Security Event Alerting	Centralize security event alerting across enterprise assets for log correlation and analysis. Best practice implementation requires the use of a SIEM, which includes vendor-defined event correlation alerts. A log analytics platform configured with security-relevant correlation alerts also satisfies this Safeguard.		●	●	Integrate SaaS model provider logs into enterprise SIEM where necessary, correlating them with internal identities and systems. SaaS logs often include visibility into rate limits, errors, or policy violations. Integration enables full end-to-end detection and response.
13.2	Deploy a Host-Based Intrusion Detection Solution	Deploy a host-based intrusion detection solution on enterprise assets, where appropriate and/or supported.		●	●	Deploy an IDS on servers hosting AI models to detect suspicious file changes, process injections, or configuration alterations. Provides early warning of attackers attempting to tamper with models, modify training data, or install persistence mechanisms directly on the host.

CIS Control 13: Network Monitoring and Defense

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
13.3	Deploy a Network Intrusion Detection Solution	Deploy a network intrusion detection solution on enterprise assets, where appropriate. Example implementations include the use of a Network Intrusion Detection System (NIDS) or equivalent cloud service provider (CSP) service.		●	●	Monitor AI-related network traffic for anomalous patterns that may indicate model extraction, brute-force prompting, or data exfiltration. Model extraction activity often involves rapid, systematic, or repetitive queries. Network monitoring tools must detect these behaviors and correlate them with identities and systems.
13.4	Perform Traffic Filtering Between Network Segments	Perform traffic filtering between network segments, where appropriate.		●	●	Filter traffic between AI training, inference, and data storage segments for Endpoint-Hosted and Enterprise-Hosted LLMs as well as between endpoint calling SaaS-Hosted Models and the SaaS services to restrict communication to only necessary flows. Limits the blast radius of a compromise, preventing an attacker in one zone from easily moving to others, thus protecting critical assets.
13.5	Manage Access Control for Remote Assets	Manage access control for assets remotely connecting to enterprise resources. Determine amount of access to enterprise resources based on: up-to-date anti-malware software installed, configuration compliance with the enterprise's secure configuration process, and ensuring the operating system and applications are up-to-date.		●	●	Enforce strict access control policies for remote devices connecting to all AI environments, and verify security posture before granting access. Prevents compromised or non-compliant remote devices from introducing threats into the secure AI network or acting as a bridge for attackers.
13.6	Collect Network Traffic Flow Logs	Collect network traffic flow logs and/or network traffic to review and alert upon from network devices.		●	●	Collect flow logs from networks hosting AI workloads to analyze traffic patterns and detect anomalies. Essential for identifying data exfiltration attempts, unusual volume spikes, and unauthorized connections to/from AI systems that may indicate a breach.
13.7	Deploy a Host-Based Intrusion Prevention Solution	Deploy a host-based intrusion prevention solution on enterprise assets, where appropriate and/or supported. Example implementations include use of an Endpoint Detection and Response (EDR) client or host-based IPS agent.			●	Deploy an EDR on all AI hosts to block malicious execution and scan model-generated code. Model-serving infrastructure (e.g., GPU clusters, inference servers) requires rigorous behavioral monitoring to detect process injection, lateral movement, or tampering with model artifacts. However, in AI environments, the malware often originates from the application itself, as LLMs can generate and attempt to execute dangerous scripts or commands via code interpreters. Therefore, host-based intrusion prevention must be tuned to not only protect the server from external attackers but also to actively scan and block unvalidated model-generated code before execution, effectively acting as a runtime gate that prevents a confused or jailbroken model from launching exploits against its own environment.
13.8	Deploy a Network Intrusion Prevention Solution	Deploy a network intrusion prevention solution, where appropriate. Example implementations include the use of a Network Intrusion Prevention System (NIPS) or equivalent CSP service.			●	Deploy an IPS to inspect traffic to and from AI environments for known exploit signatures and malicious payloads. An IPS actively blocks network-based attacks against LLM vulnerabilities before they can reach and compromise the target systems.
13.9	Deploy Port-Level Access Control	Deploy port-level access control. Port-level access control utilizes 802.1x, or similar network access control protocols, such as certificates, and may incorporate user and/or device authentication.			●	No Additional AI LLM Guidance
13.10	Perform Application Layer Filtering	Perform application layer filtering. Example implementations include a filtering proxy, application layer firewall, or gateway.			●	Use application-layer filtering to inspect and control traffic protocols used by LLM services (e.g., restricting API calls). This prevents misuse of allowed protocols and blocks attacks that tunnel through standard ports to reach AI applications, enhancing defense depth.
13.11	Tune Security Event Alerting Thresholds	Tune security event alerting thresholds monthly, or more frequently.			●	Tune alert thresholds for AI-specific network traffic to reduce false positives and ensure timely detection of genuine threats. AI traffic patterns can be unique and are changing very fast. Proper tuning ensures security teams aren't overwhelmed by noise while still catching real attacks effectively.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local LLM tools using network access should be monitored through endpoint firewalls and DNS filtering to detect unusual outbound patterns.
- Tools that generate HTTP requests, such as AI-based browsing assistants, must route traffic through enterprise egress points for inspection.
- Local inference tools should not establish direct connections to internal model stores without authorization.

Enterprise-Hosted Models

- Training clusters often produce predictable high-bandwidth traffic; deviations from expected patterns may indicate compromise.
- Inference endpoints should be placed behind secured gateways where traffic inspection, rate limiting, and anomaly detection are enforced.
- Vector stores or RAG systems must be monitored for signs of poisoning, abnormal read/write patterns, or unauthorized cross-zone access.
- Internal orchestration systems should log and expose traffic between components for centralized monitoring.

SaaS-Hosted Models Accessed via API

- Enterprises should monitor outbound traffic to SaaS model domains for volume anomalies, data spikes, or suspicious timing.
- Client applications should be restricted to specific provider domains and subdomains. DNS monitoring should detect shadow or impersonating endpoints.
- Provider-generated telemetry (e.g., rate-limit warnings, safety alerts, or behavior blocks) should feed into network alerting pipelines.

Additional AI LLM Considerations

- Inference APIs can become high-value targets for brute-force or automated misuse; traffic analytics should detect “low-and-slow” extraction attempts.
- AI systems performing autonomous or semi-autonomous web browsing must have their traffic scrutinized, including outbound HTTP requests triggered by model logic.
- AI-related incidents (e.g., poisoning through RAG pipelines) often manifest first as strange network patterns, such as unexpected connections between inference nodes and document sources or nonstandard query sizes.
- For multimodal models, binary data uploads and downloads may bypass normal text-based inspection. Monitoring tools should treat these traffic types as high-risk.
- If AI agents interact with internal business systems (e.g., ERP, CRM), network monitoring must ensure those interactions follow approved patterns and cannot be redirected to external endpoints.

Control 14: Security Awareness and Skills Training

Establish and maintain a security awareness program to influence behavior among the workforce to be security conscious and properly skilled to reduce cybersecurity risks to the enterprise.

AI LLM Applicability

LLM systems introduce unfamiliar risks for many users that traditional security training does not address. Users may inadvertently expose sensitive data by pasting it into public AI tools, misunderstanding how prompts and completions are stored, logged, or reused. Developers may integrate models incorrectly, over-trust model output, or fail to establish guardrails. Data scientists may unintentionally create unsafe fine-tuning datasets or mishandle high-sensitivity training data. Security teams may overlook AI patterns that differ from traditional threats, such as prompt-injection attempts, model extraction behaviors, or indirect data leakage.

Training must address these gaps by helping personnel understand:

- How LLMs operate and why natural-language systems are inherently sensitive to manipulation
- Which data can and cannot be shared with different types of AI services
- The unique risks posed by model outputs, including hallucination, leakage, or malicious code
- The security implications of retrieval pipelines, embeddings, and memory
- How authorization must never be delegated to the model itself
- How adversaries may exploit AI systems to extract data, bypass controls, or manipulate logic
- The importance of safe integration patterns and avoiding excessive trust in model responses

Since AI systems touch a broad variety of roles including business users, customer support, developers, data scientists, analysts, and IT/SecOps staff, training must be tailored so that each group receives actionable guidance relevant to their responsibilities.

Safeguards

CIS Control 14: Security Awareness and Skills Training

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
14.1	Establish and Maintain a Security Awareness Program	Establish and maintain a security awareness program. The purpose of a security awareness program is to educate the enterprise's workforce on how to interact with enterprise assets and data in a secure manner. Conduct training at hire and, at a minimum, annually. Review and update content annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Incorporate AI-specific risks, such as prompt injection and data leakage, into the enterprise security awareness program. Educated users are the first line of defense; understanding AI risks helps prevent accidental data exposure and misuse of AI tools.
14.2	Train Workforce Members to Recognize Social Engineering Attacks	Train workforce members to recognize social engineering attacks, such as phishing, business email compromise (BEC), pretexting, and tailgating.	●	●	●	Train staff to recognize social engineering attacks that leverage AI-generated content, such as deepfakes or convincing phishing emails. AI tools make social engineering more sophisticated; specialized training is needed to help employees identify these advanced threats and avoid manipulation.
14.3	Train Workforce Members on Authentication Best Practices	Train workforce members on authentication best practices. Example topics include MFA, password composition, and credential management.	●	●	●	No Additional AI LLM Guidance

CIS Control 14: Security Awareness and Skills Training

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
14.4	Train Workforce on Data Handling Best Practices	Train workforce members on how to identify and properly store, transfer, archive, and destroy sensitive data. This also includes training workforce members on clear screen and desk best practices, such as locking their screen when they step away from their enterprise asset, erasing physical and virtual whiteboards at the end of meetings, and storing data and assets securely.	●	●	●	Train users not to enter sensitive, regulated, or proprietary data into unapproved AI systems or public AI chat interfaces. Users must understand the differences between enterprise-approved models and consumer-facing tools, and how data retention or reuse in those systems may violate policy.
14.5	Train Workforce Members on Causes of Unintentional Data Exposure	Train workforce members to be aware of causes for unintentional data exposure. Example topics include mis-delivery of sensitive data, losing a portable end-user device, or publishing data to unintended audiences.	●	●	●	Educate users and enforce policy prohibiting entry of sensitive or regulated data into unapproved AI web interfaces. Policies must clearly differentiate between sanctioned enterprise AI endpoints and consumer AI websites, and users must understand that “just trying something” in a public AI chat can constitute a data breach.
14.6	Train Workforce Members on Recognizing and Reporting Security Incidents	Train workforce members to be able to recognize a potential incident and be able to report such an incident.	●	●	●	Train employees to recognize and report AI-specific security incidents, such as unexpected model behavior or potential data leaks. Rapid reporting enables faster incident response, minimizing the damage caused by AI-related security breaches.
14.7	Train Workforce on How to Identify and Report if Their Enterprise Assets are Missing Security Updates	Train workforce to understand how to verify and report out-of-date software patches or any failures in automated processes and tools. Part of this training should include notifying IT personnel of any failures in automated processes and tools.	●	●	●	Educate staff on the importance of keeping AI tools and local environments updated and how to report issues. Ensures that vulnerabilities in decentralized AI tools, such as local LLM runners, are identified and addressed promptly, reducing the attack surface.
14.8	Train Workforce on the Dangers of Connecting to and Transmitting Enterprise Data Over Insecure Networks	Train workforce members on the dangers of connecting to, and transmitting data over, insecure networks for enterprise activities. If the enterprise has remote workers, training must include guidance to ensure that all users securely configure their home network infrastructure.	●	●	●	Train users on the risks of accessing AI services or transmitting training data over unsecured public networks. Prevents interception of sensitive AI data and credentials when employees are working remotely or traveling, thus protecting confidentiality.
14.9	Conduct Role-Specific Security Awareness and Skills Training	Conduct role-specific security awareness and skills training. Example implementations include secure system administration courses for IT professionals, OWASP® Top 10 vulnerability awareness and prevention training for web application developers, and advanced social engineering awareness training for high-profile roles.		●	●	Train personnel on role-specific AI risks, including data poisoning, safe ingestion pipelines, and recognizing adversarial telemetry. Security awareness must target the specific hazards of the AI life cycle. Data curators and pipeline engineers need training on the risks of “auto-ingestion” and poisoning, understanding that pulling untrusted content or managing unvetted datasets can permanently corrupt the model’s integrity or introduce backdoors. Simultaneously, security operations teams require specialized training to interpret AI-specific telemetry, learning to distinguish between benign usage and subtle behavioral indicators like extraction attempts, prompt injection campaigns, or suspicious vector store access, ensuring they can effectively monitor and defend the unique attack surface of generative systems.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Users running local LLM tools must understand the risks of caching, local storage, and output logs that may contain sensitive data.
- Training should emphasize that local tools do not necessarily inherit enterprise protections and may retain data indefinitely.
- Developers using notebooks should be trained to avoid accidental exposure of secrets, access tokens, or datasets during experimentation.

Enterprise-Hosted Models

- Training for ML engineers and infrastructure teams should cover safe operation of training clusters, GPU nodes, vector stores, and model registries.
- Teams must understand how to prevent accidental cross-environment data transfers (development → production, staging → inference).
- Operations staff should be trained to review logs for AI-specific indicators and to apply security controls across distributed AI environments.

SaaS-Hosted Models Accessed via API

- Users must be trained on which SaaS models are approved, what data can be sent to them, and how contractual restrictions apply.
- Developers and analysts integrating SaaS APIs must be trained on safely handling API keys, understanding retention settings, and avoiding over-sharing data in prompts.
- Teams must know how to interpret provider warnings or rate-limit events that may indicate misuse or attempted attacks.

Additional AI LLM Considerations

- Training should emphasize not over-trusting model outputs, particularly for code, compliance, legal, or security-relevant content.
- For environments with tool-enabled models (e.g., browsing, code execution), training must highlight that prompts alone should never authorize actions.
- Teams should understand how “harmless” content (e.g., summaries, rewritten text) may still inadvertently leak sensitive details.
- Memory and RAG systems introduce new risks: staff must understand their persistence, sensitivity, and susceptibility to poisoning or misclassification.
- Training for executives and managers should cover AI governance fundamentals, including risk boundaries, life cycle responsibilities, and shared responsibility across hosting types.

Control 15: Service Provider Management

Develop a process to evaluate service providers who hold sensitive data, or are responsible for an enterprise’s critical IT platforms or processes, to ensure these providers are protecting those platforms and data appropriately.

AI LLM Applicability

AI systems frequently depend on external providers, such as SaaS-hosted model APIs, managed vector database services, embedding-as-a-service platforms, or vendor-operated fine-tuning environments. These providers may handle sensitive enterprise data, store prompts and outputs, maintain logs, or apply internal safety and retention policies that differ from the enterprise’s expectations.

Unique risks introduced by AI service providers include:

- Opaque data handling practices, including training-data reuse, prompt logging, long-term retention, or co-mingled processing
- Rapid model updates, where a provider replaces or modifies underlying models, changing behavior unexpectedly
- Cross-tenant leakage risks, where another customer’s activity may influence model behavior or inference results
- Limited transparency around training data lineage, model versions, or operational security controls
- Potentially unsafe modalities, such as image or audio inputs, that require additional controls
- Concentration risk, when enterprises rely heavily on a single external AI provider for critical functions

SaaS AI providers often operate as high-privilege processing environments, and many expose powerful capabilities (e.g., tool invocation, code execution, search functions) that can affect downstream systems if misused. Effective service provider management ensures that contractual, operational, and technical requirements align with the enterprise’s security needs, data classification rules, and regulatory obligations.

Safeguards

CIS Control 15: Service Provider Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
15.1	Establish and Maintain an Inventory of Service Providers	Establish and maintain an inventory of service providers. The inventory is to list all known service providers, include classification(s), and designate an enterprise contact for each service provider. Review and update the inventory annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Maintain a catalog of approved AI service providers. Clear documentation prevents accidental use of unapproved or insecure providers.
15.2	Establish and Maintain a Service Provider Management Policy	Establish and maintain a service provider management policy. Ensure the policy addresses the classification, inventory, assessment, monitoring, and decommissioning of service providers. Review and update the policy annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	The service provider management policy must enforce a “conservative default” posture by mandating a safe baseline: disabling history retention, opting out of model training, pinning specific model versions, and disabling risky capabilities (e.g., agents) until validated. Policies must also require evidence of tenant isolation, logging clarity, and AI-specific incident notification SLAs. Permissive SaaS defaults may expose sensitive data for training or introduce risks via unmanaged feature updates and code execution. Enforcing these requirements, alongside data residency checks and vendor diversification, ensures that SaaS inference does not compromise data sovereignty, regulatory compliance, or operational resilience.

CIS Control 15: Service Provider Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
15.3	Classify Service Providers	Classify service providers. Classification consideration may include one or more characteristics, such as data sensitivity, data volume, availability requirements, applicable regulations, inherent risk, and mitigated risk. Update and review classifications annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Classify each approved AI service provider by authorized data classifications. Classification ensures alignment between sensitivity levels and provider assurances.
15.4	Ensure Service Provider Contracts Include Security Requirements	Ensure service provider contracts include security requirements. Example requirements may include minimum security program requirements, security incident and/or data breach notification and response, data encryption requirements, and data disposal commitments. These security requirements must be consistent with the enterprise's service provider management policy. Review service provider contracts annually to ensure contracts are not missing security requirements.		●	●	Mandate contractual guarantees for data residency, retention limits, and strict model version transparency. Standard cloud agreements often overlook the specific risks of generative AI. Contracts must explicitly enforce data sovereignty and retention limits to prevent providers from silently harvesting sensitive prompts for training or storing regulated data in non-compliant jurisdictions. Equally critical is the requirement for operational transparency regarding model versions. Unlike static software, LLMs are prone to "behavioral drift" where provider updates can inadvertently degrade performance or bypass established safety guardrails. Without contractual rights to pin model versions or receive advance notice of changes, the enterprise risks sudden application failure or security regression. Thus, legal terms must govern not only data privacy but also the stability and integrity of the inference engine itself.
15.5	Assess Service Providers	Assess service providers consistent with the enterprise's service provider management policy. Assessment scope may vary based on classification(s), and may include review of standardized assessment reports, such as Service Organization Control 2 (SOC 2) and Payment Card Industry (PCI) Attestation of Compliance (AoC), customized questionnaires, or other appropriately rigorous processes. Reassess service providers annually, at a minimum, or with new and renewed contracts.			●	Evaluate providers for rigorous data isolation, tenant segregation, and cross-modality safety defenses. Standard vendor assessments often miss AI-specific risks. Evaluations must verify that providers enforce strict multi-tenant isolation to prevent "data bleed-through" or inference cache contamination between customers. As models become multi-modal, providers must be assessed based on their ability to isolate multimodal inputs and prevent them from hijacking the model's text-based system instructions. The assessment must confirm that the provider's architecture defends against both lateral data leakage across tenants and vertical privilege escalation via non-text injection vectors.
15.6	Monitor Service Providers	Monitor service providers consistent with the enterprise's service provider management policy. Monitoring may include periodic reassessment of service provider compliance, monitoring service provider release notes, and dark web monitoring.			●	Continuously monitor AI service providers for changes in security posture, terms of service, or data handling practices. Ensures ongoing compliance with enterprise security requirements and allows for timely reaction to provider-side risks that could impact AI operations.
15.7	Securely Decommission Service Providers	Securely decommission service providers. Example considerations include user and service account deactivation, termination of data flows, and secure disposal of enterprise data within service provider systems.			●	Establish a process for securely removing data and revoking access when terminating AI service provider contracts. Prevents residual data exposure and ensures that former providers no longer have access to enterprise AI assets or proprietary information.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Providers may supply local runtimes or model weights; enterprises must ensure that licenses, update mechanisms, and data-sharing behaviors align with enterprise policy.
- Local tools that call cloud-hosted endpoints must still follow provider management rules. Developers should not integrate personal accounts or consumer-model APIs into enterprise workflows.

Enterprise-Hosted Models

- Even when models are self-hosted, enterprises may depend on third-party model repositories, driver distributions, MLOps platforms, or embedding services; each must undergo provider evaluation.
- Hybrid setups (local inference + cloud analytics) require careful governance to ensure that data flowing to cloud components adheres to classification and residency rules.

SaaS-Hosted Models Accessed via API

- Provider responsibilities include runtime security, model patching, isolation, and operational availability. Enterprises must verify that these align with enterprise requirements.
- Contracts should specify whether fine-tuning data is stored, for how long, and whether it may be used to improve provider services.
- Access to provider dashboards and admin portals must integrate with enterprise SSO and logging.

Additional AI LLM Considerations

- Providers offering multimodal capabilities may store image or audio data differently from text. Enterprises must understand these distinctions before enabling them.
- Providers that continuously retrain or improve their models may introduce unpredictable behavior unless version control or freezing mechanisms are available.
- AI service providers often hold logs, embeddings, or metadata that can indirectly reveal sensitive enterprise information and must be addressed in contracts and risk assessments.
- Enterprises should periodically re-evaluate provider risk as the AI ecosystem evolves, new models are released, or regulatory requirements change.
- Independent audits or third-party certifications (e.g., SOC 2, ISO 27001, and more) provide baseline validation but should not replace direct assessment of AI-specific risks.

Control 16: Application Software Security

Manage the security life cycle of in-house developed, hosted, or acquired software to prevent, detect, and remediate security weaknesses before they can impact the enterprise.

AI LLM Applicability

LLM-enabled applications introduce new classes of risks that traditional Software Development Lifecycle (SDLC) controls do not inherently address. Modern applications increasingly rely on LLMs to generate content, interpret instructions, make decisions, route requests, retrieve documents, call tools, or produce code. These capabilities can profoundly affect application logic, data flows, and operational security.

Application-layer risks specific to LLM systems include:

- Prompt injection, where untrusted content modifies the behavior of system prompts, functions, or tools.
- Output misuse, where applications trust model output without validation, causing incorrect or unsafe downstream actions.
- Unsafe tool invocation, where LLMs trigger high-impact actions (e.g., file writes, HTTP requests, code execution) based solely on natural-language prompts.
- RAG poisoning, where document retrieval feeds adversarial or misleading content into the model context. Note: While RAG is not in scope for this guide, we note the risk for completeness.
- Data contamination, where untrusted inputs enter fine-tuning pipelines or memory stores.
- Behavioral drift, where model updates or retraining causes inconsistent or unexpected output.
- Insufficient isolation, where applications allow LLMs to mix content, instructions, and user data in uncontrolled ways.

To address these risks, enterprises must embed AI-specific best practices into design, coding practices, integration testing, runtime controls, and ongoing monitoring. LLM integration should never bypass traditional application security; instead, application-layer logic becomes even more critical, acting as the “policy spine” that constrains and interprets model behavior safely.

Safeguards

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
16.1	Establish and Maintain a Secure Application Development Process	Establish and maintain a secure application development process. In the process, address such items as: secure application design standards, secure coding practices, developer training, vulnerability management, security of third-party code, and application security testing procedures. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Integrate security controls into the life cycle of AI application development, from model selection to hosting decisions to deployment. Build security in from the start, reducing the likelihood of vulnerabilities in the final AI-enabled application and lowering long-term maintenance costs.
16.2	Establish and Maintain a Process to Accept and Address Software Vulnerabilities	Establish and maintain a process to accept and address reports of software vulnerabilities, including providing a means for external entities to report. The process is to include such items as: a vulnerability handling policy that identifies reporting process, responsible party for handling vulnerability reports, and a process for intake, assignment, remediation, and remediation testing. As part of the process, use a vulnerability tracking system that includes severity ratings and metrics for measuring timing for identification, analysis, and remediation of vulnerabilities. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard. Third-party application developers need to consider this an externally-facing policy that helps to set expectations for outside stakeholders.		●	●	Monitor for published model/family vulnerabilities and update inventories accordingly. Vulnerabilities affecting specific model families, tokenizers, or serving frameworks must be tracked alongside traditional Common Vulnerabilities and Exposures (CVEs) to ensure timely mitigation.

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
16.3	Perform Root Cause Analysis on Security Vulnerabilities	Perform root cause analysis on security vulnerabilities. When reviewing vulnerabilities, root cause analysis is the task of evaluating underlying issues that create vulnerabilities in code, and allows development teams to move beyond just fixing individual vulnerabilities as they arise.		●	●	Conduct adversarial evaluation of AI features, including testing for prompt injection, content leakage, jailbreak attempts, and unsafe output generation. AI systems require specialized security testing to identify model-specific weaknesses that normal functional testing would not reveal.
16.4	Establish and Manage an Inventory of Third-Party Software Components	Establish and manage an updated inventory of third-party components used in development, often referred to as a “bill of materials,” as well as components slated for future use. This inventory is to include any risks that each third-party component could pose. Evaluate the list at least monthly to identify any changes or updates to these components, and validate that the component is still supported.		●	●	Maintain an AI-Bill of Materials (BOM)/Model BOM documenting provenance, integrity, and dependencies of all model artifacts. A Model BOM allows enterprises to understand upstream dependencies, verify authenticity, and trace vulnerabilities or harmful behavior back to specific sources or versions.
16.5	Use Up-to-Date and Trusted Third-Party Software Components	Use up-to-date and trusted third-party software components. When possible, choose established and proven frameworks and libraries that provide adequate security. Acquire these components from trusted sources or evaluate the software for vulnerabilities before use.		●	●	Enforce strict integrity verification for all training data, model weights, and runtime components. In the AI supply chain, data functions as code. Using unvetted datasets invites “poisoning” attacks where adversaries embed backdoors directly into model logic. Equally critical is the runtime environment; model weights and custom operators (e.g., PyTorch pickles) are often capable of executing arbitrary code upon loading. Therefore, the SDLC must enforce a rigorous “chain of trust” that validates the cryptographic integrity and provenance of every asset, from the raw fine-tuning corpora to the final inference binaries, ensuring that the application never loads compromised artifacts or malicious dynamic kernels.
16.6	Establish and Maintain a Severity Rating System and Process for Application Vulnerabilities	Establish and maintain a severity rating system and process for application vulnerabilities that facilitates prioritizing the order in which discovered vulnerabilities are fixed. This process includes setting a minimum level of security acceptability for releasing code or applications. Severity ratings bring a systematic way of triaging vulnerabilities that improves risk management and helps ensure the most severe bugs are fixed first. Review and update the system and process annually.		●	●	Establish a system for rating the severity of vulnerabilities in AI models and applications to prioritize remediation. This ensures that the most critical AI risks are addressed first, optimizing the use of limited security resources and reducing exposure.
16.7	Use Standard Hardening Configuration Templates for Application Infrastructure	Use standard, industry-recommended hardening configuration templates for application infrastructure components. This includes underlying servers, databases, and web servers, and applies to cloud containers, Platform as a Service (PaaS) components, and SaaS components. Do not allow in-house developed software to weaken configuration hardening.		●	●	Use hardened configuration templates for the infrastructure supporting AI applications (e.g., servers, containers, databases). Reduces the attack surface of the underlying infrastructure, making it harder for attackers to compromise the AI application or its environment.
16.8	Separate Production and Non-Production Systems	Maintain separate environments for production and non-production systems.		●	●	Isolate development, test, staging, and production environments, with defined steps for reviewing and approving changes before they move into production. Separation prevents accidental exposure of sensitive data in development/testing environments and prevents unverified models from entering production without review.
16.9	Train Developers in Application Security Concepts and Secure Coding	Ensure that all software development personnel receive training in writing secure code for their specific development environment and responsibilities. Training can include general security principles and application security standard practices. Conduct training at least annually and design in a way to promote security within the development team, and build a culture of security among the developers.		●	●	Train builders on AI-specific attack vectors and secure integration patterns, emphasizing guardrails and validation. Developers and data scientists often treat models as trusted components, unaware that input channels are vectors for prompt injection and output streams can carry malicious payloads. Technical staff must be trained to recognize these model-specific threats, such as extraction and leakage, and to implement “defensive programming” for AI. This includes mastering safe integration patterns where model outputs are never trusted implicitly but are instead subject to rigorous input validation, output filtering, and business logic guardrails. Without this specialized training, naive integrations will lack the necessary verification layers to prevent a compromised model from manipulating downstream systems.

CIS Control 16: Application Software Security

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
16.10	Apply Secure Design Principles in Application Architectures	Apply secure design principles in application architectures. Secure design principles include the concept of least privilege and enforcing mediation to validate every operation that the user makes, promoting the concept of “never trust user input.” Examples include ensuring that explicit error checking is performed and documented for all input, including for size, data type, and acceptable ranges or formats. Secure design also means minimizing the application infrastructure attack surface, such as turning off unprotected ports and services, removing unnecessary programs and files, and renaming or removing default accounts.		●	●	Enforce strict architectural separation between privileged system instructions and untrusted inputs or retrieved data. Design applications to treat system prompts as immutable code, using strict template binding to ensure user inputs are sandboxed as passive data variables. Applications must never rely on the probabilistic judgment of an LLM to determine if an action is authorized; authorization must be enforced by deterministic application logic external to the model. Secure design also requires stateful validation across multi-turn sessions to prevent “boiling frog” attacks, where a benign initial request evolves into a prohibited action. When model behavior is ambiguous or violates policy, the system must default to a “deny” state rather than attempt to autocorrect.
16.11	Leverage Vetted Modules or Services for Application Security Components	Leverage vetted modules or services for application security components, such as identity management, encryption, auditing, and logging. Using platform features in critical security functions will reduce developers' workload and minimize the likelihood of design or implementation errors. Modern operating systems provide effective mechanisms for identification, authentication, and authorization and make those mechanisms available to applications. Use only standardized, currently accepted, and extensively reviewed encryption algorithms. Operating systems also provide mechanisms to create and maintain secure audit logs.		●	●	Implement vetted, code-based safety frameworks (e.g., NeMo Guardrails, Llama Guard) to enforce security policies, rather than relying solely on system prompts or natural language instructions as security controls. System prompts are easily bypassed via injection and lack deterministic enforcement; external, vetted safety modules provide a necessary layer of verifiable, code-level defense against adversarial inputs.
16.12	Implement Code-Level Security Checks	Apply static and dynamic analysis tools within the application life cycle to verify that secure coding practices are being followed.			●	Validate model artifacts through signatures or checksums before deployment. Models and related files must not be loaded from unverified or tampered sources. Integrity validation ensures training or inference uses only approved artifacts.
16.13	Conduct Application Penetration Testing	Conduct application penetration testing. For critical applications, authenticated penetration testing is better suited to finding business logic vulnerabilities than code scanning and automated security testing. Penetration testing relies on the skill of the tester to manually manipulate an application as an authenticated and unauthenticated user.			●	Test for multimodal injection vectors and validate application fail-safes against malicious model outputs. Standard penetration testing often misses the unique “blind spots” of generative AI. Testers must explicitly target the model's perception layers using adversarial images (visual jailbreaks) and steganographic audio commands to determine if non-text inputs can bypass textual safety guardrails. However, testing the input is only half the battle; the assessment must also validate the application's output handling. It is critical to confirm that if a model is successfully duped into generating malicious code or policy-violating text, the downstream application logic correctly identifies the unsafe payload and triggers a fail-safe “deny” state, rather than blindly executing the compromised instruction.
16.14	Conduct Threat Modeling	Conduct threat modeling. Threat modeling is the process of identifying and addressing application security design flaws within a design, before code is created. It is conducted through specially trained individuals who evaluate the application design and gauge security risks for each entry point and access level. The goal is to map out the application, architecture, and infrastructure in a structured way to understand its weaknesses.			●	Perform threat modeling specifically for AI systems to identify unique attack vectors like adversarial examples or inversion attacks. Proactively identifies design flaws and security gaps in AI systems before they can be exploited in production, improving overall system resilience.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Applications embedding local models must ensure that model prompts, outputs, and tool calls do not bypass OS-level or application-level controls.
- Developers using notebooks or local inference tools should integrate guardrails (sanitization, validation) into their environment, not assume model output is safe.
- Endpoints using local LLMs must enforce separation between local files and model-driven actions.

Enterprise-Hosted Models

- Model-serving APIs should be accessed through authenticated, authorized, and audited application service accounts.
- Application guardrails must be deployed consistently across distributed inference nodes to prevent inconsistent behavior.
- Fine-tuning pipelines must integrate into CI/CD, ensuring that training data, scripts, and configuration files cannot be modified without review.

SaaS-Hosted Models Accessed via API

- Applications calling SaaS models must validate provider responses and cannot rely on provider-internal safety layers as the sole control mechanism.
- Where SaaS providers automatically change model versions or capabilities, applications must validate compatibility and re-test guardrails following changes.
- Tool-enabled SaaS models require explicit application-layer policy enforcement around permitted and prohibited actions.

Additional AI LLM Considerations

- Treat model output as untrusted input, even when the model is hosted internally.
- Structured prompt templates reduce the likelihood of instruction/role confusion and should be used wherever possible.
- Implement architectural separation via delimiters. This will sanitize inputs by enforcing structural separation (e.g., System vs. User roles) to isolate untrusted data from model instructions. Use API-level role separation (e.g., ChatML system vs. user blocks) or distinct markers (e.g., XML tags) to prevent the model from interpreting user content as system commands. This technical implementation neutralizes semantic persuasion attacks that bypass traditional keyword syntax filters.
- Isolate non-text inputs (multimodal) from privileged tools. Images and audio must be treated as highly untrusted input vectors, as they can carry “visual jailbreaks” or hidden commands that are invisible to human reviewers and bypass text-centric filters.
- RAG pipelines must sanitize untrusted external content before passing it to models; otherwise, RAG becomes a high-risk injection vector. Note: While RAG is not in scope for this guide, we note the risk here for completeness.
- Treat retrieved text used in RAG as untrusted model input requiring classification and protection. When retrieved text is added to a prompt, treat it as untrusted input and enforce the same classification, redaction, and sanitization used for user prompts to prevent indirect injection.
- Validate and quarantine automatically ingested external content before reuse in retrieval or memory workflows. Emails, uploaded documents, or scraped content should not automatically enter high-trust AI stores. Automated validation reduces the chance of embedding harmful or deceptive content into the system.
- Treat model-generated code, configurations, or files as untrusted until validated via scanning, sandboxing, or code review. Models should not be implicitly trusted as secure code generators. Validation must occur before any generated output interacts with production systems or sensitive data.
- Outputs inserted directly into business systems (e.g., notes, tickets, emails) must be validated and enriched with metadata indicating they originated from an AI system.
- Ensure all model outputs are contextually encoded (HTML, JSON, SQL) before processing by downstream agents or browsers.

- Evaluation and testing should include adversarial content sourced from internal red teams or curated datasets reflecting known real-world exploit styles.
- Guardrail logic must be version-controlled and tested in concert with model updates, ensuring that behavioral drift does not compromise enforcement.
- Monitor for published weaknesses or instability patterns associated with specific models or model families and adjust guardrails or mitigations accordingly. Model-level weaknesses, including known jailbreak vectors or content-safety gaps, must be treated as vulnerabilities requiring mitigation rather than user education alone.
- Implement “Fail-Early” logic in LLM gateways and other control layers proxying LLM interactions. To prevent “Denial of Wallet” attacks, which aim to exhaust token budgets, the application must validate token counts, enforce strict “max_token” limits, and identify recursive loops before requests are forwarded to the model inference layer. Network controls cannot detect these semantic costs.

Control 17: Incident Response Management

Establish a program to develop and maintain an incident response capability (e.g., policies, plans, procedures, defined roles, training, and communications) to prepare, detect, and quickly respond to an attack.

AI LLM Applicability

LLM systems introduce new types of incidents, new sources of signals, and new response paths. Traditional Incident Response (IR) teams may not initially recognize AI-specific attacks or failures because they differ from conventional malware, phishing, or network intrusion patterns. A robust IR program must account for:

- Unexpected or harmful model outputs
- Prompt injection events, especially those that lead to policy bypass or unsafe actions
- Data leakage incidents, including inadvertent exposure through prompts, completions, embeddings, or logs
- Model extraction attempts, such as high-volume structured querying
- Poisoning or contamination of training data, fine-tuning sources, retrieval corpora, or persistent memory
- Compromised tool-invocation pathways, where an attacker influences an LLM to trigger external actions
- Behavioral drift, where model performance degrades or becomes unsafe following an update
- Unauthorized model updates or deployments
- SaaS provider-originated incidents, where model behavior changes unexpectedly due to provider-side updates

Incident response for AI systems must be capable of isolating affected environments (training clusters, inference nodes, vector stores), rolling back model versions, revoking compromised API keys, invalidating unsafe memory or retrieval data, and coordinating tightly with providers. Clear AI-specific playbooks, logging, and escalation paths are essential.

Safeguards

CIS Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
17.1	Designate Personnel to Manage Incident Handling	Designate one key person, and at least one backup, who will manage the enterprise's incident handling process. Management personnel are responsible for the coordination and documentation of incident response and recovery efforts and can consist of employees internal to the enterprise, service providers, or a hybrid approach. If using a service provider, designate at least one person internal to the enterprise to oversee any third-party work. Review annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Assign specific personnel with AI expertise to manage incidents involving LLMs and generative AI systems. AI incidents require specialized knowledge to diagnose and mitigate; designated experts ensure an effective and informed response.
17.2	Establish and Maintain Contact Information for Reporting Security Incidents	Establish and maintain contact information for parties that need to be informed of security incidents. Contacts may include internal staff, service providers, law enforcement, cyber insurance providers, relevant government agencies, Information Sharing and Analysis Center (ISAC) partners, or other stakeholders. Verify contacts annually to ensure that information is up-to-date.	●	●	●	Maintain up-to-date contact information for reporting AI-related security incidents to internal teams and external AI providers. Facilitates rapid communication during a crisis, ensuring that the right people are alerted immediately to contain the incident.

CIS Control 17: Incident Response Management

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
17.3	Establish and Maintain an Enterprise Process for Reporting Incidents	Establish and maintain a documented enterprise process for the workforce to report security incidents. The process includes reporting timeframe, personnel to report to, mechanism for reporting, and the minimum information to be reported. Ensure the process is publicly available to all of the workforce. Review annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●	Define a clear process for employees to report suspected AI security incidents, including specific indicators to look for. Encourages timely reporting of anomalies, enabling the security team to investigate and contain threats earlier in the attack life cycle.
17.4	Establish and Maintain an Incident Response Process	Establish and maintain a documented incident response process that addresses roles and responsibilities, compliance requirements, and a communication plan. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Define incident response procedures that address AI-specific attack scenarios, such as unsafe model behavior, leakage, extraction attempts, poisoning, or compromised tools. IR plans must explicitly identify how to detect and respond to incidents involving LLMs, including the unique signals and artifacts (prompts, outputs, logs, embeddings, training data, model versions) associated with such events.
17.5	Assign Key Roles and Responsibilities	Assign key roles and responsibilities for incident response, including staff from legal, IT, information security, facilities, public relations, human resources, incident responders, analysts, and relevant third parties. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Define specific roles and responsibilities for AI incident response, including data scientists, legal representatives, and privacy officers. Ensures a coordinated effort where all aspects of an AI incident (e.g., technical, legal, reputational) are addressed efficiently by the appropriate stakeholders.
17.6	Define Mechanisms for Communicating During Incident Response	Determine which primary and secondary mechanisms will be used to communicate and report during a security incident. Mechanisms can include phone calls, emails, secure chat, or notification letters. Keep in mind that certain mechanisms, such as emails, can be affected during a security incident. Review annually, or when significant enterprise changes occur that could impact this Safeguard.		●	●	Establish secure communication channels for discussing AI incidents, especially those involving sensitive data leaks. Prevents attackers from intercepting incident response communications and ensures stakeholders are kept informed securely without tipping off the adversary.
17.7	Conduct Routine Incident Response Exercises	Plan and conduct routine incident response exercises and scenarios for key personnel involved in the incident response process to prepare for responding to real-world incidents. Exercises need to test communication channels, decision making, and workflows. Conduct testing on an annual basis, at a minimum.		●	●	Conduct AI-inclusive incident response exercises to validate readiness and clarify cross-team responsibilities. Exercises should incorporate realistic model-specific threats (e.g., injection attempts, data contamination, misuse of tool invocation) and help teams practice coordination between security, data science, and application groups.
17.8	Conduct Post-Incident Reviews	Conduct post-incident reviews. Post-incident reviews help prevent incident recurrence through identifying lessons learned and follow-up action.		●	●	Conduct post-incident reviews focusing on data sources, model changes, prompt changes, policy changes, tool usage, and gaps in monitoring. These reviews should mandate that these assets, at a minimum, are preserved for review: ▪ Prompts/policies versions ▪ Model versions and hashes ▪ RAG corpus snapshots ▪ Vector database logs/snapshots ▪ Gateway logs ▪ Provider admin/audit logs
17.9	Establish and Maintain Security Incident Thresholds	Establish and maintain security incident thresholds, including, at a minimum, differentiating between an incident and an event. Examples can include: abnormal activity, security vulnerability, security weakness, data breach, privacy incident, etc. Review annually, or when significant enterprise changes occur that could impact this Safeguard.			●	Enterprises should define clear security incident thresholds that distinguish normal model behavior from unsafe or compromised behavior and tie those thresholds to rapid-disable mechanisms. This includes identifying what constitutes an “event” versus an “incident” in an LLM context. For example, isolated abnormal outputs, unexpected tool calls, or minor prompt-handling issues may be events, while sustained unsafe outputs, attempts to take unapproved actions through tools, or indications of compromised model behavior should be treated as incidents. When those thresholds are met, enterprises need kill switches or rapid-disable mechanisms for models, endpoints, or tool-enabled capabilities, especially if models are producing unsafe outputs or taking unapproved actions.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Incident response must include the ability to disable or quarantine local LLM clients, especially if sensitive data exposure or unsafe code generation is suspected.
- Local logs, caches, or model artifacts may be needed for forensic review; endpoint visibility tools must capture these artifacts securely.
- Revoking compromised API keys on endpoints must be quick and centrally managed.

Enterprise-Hosted Models

- Inference servers and training clusters should support rapid isolation through network segmentation or stopping services.
- IR teams must be able to identify which model version was active during an incident, and which datasets influenced behavior.
- Internal tool ecosystems (e.g., document retrieval, memory storage, processing pipelines) may need coordinated containment, such as clearing memory, disabling ingestion flows, or quarantining retrieval indexes.

SaaS-Hosted Models Accessed via API

- Incident response may require coordination with the provider when model behavior changes unexpectedly or when logs indicate suspicious activity.
- Enterprises must be able to revoke compromised API keys immediately and disable endpoints without waiting for provider-side changes.
- SaaS-provider notifications (e.g., rate-limit warnings, safety blocks, anomaly reports) must feed into IR processes as potential incident triggers.

Additional AI LLM Considerations

- IR playbooks should include prompt and output capture procedures, which are critical for diagnosing AI incidents.
- Memory-enabled models require clear processes for memory purge during incident handling.
- Retrieval pipelines may need content quarantine and re-indexing after poisoning incidents.
- Rollback procedures must consider not just restoring data, but restoring behavioral fidelity to ensure the model acts as it did prior to the incident.
- If AI agents or tool-using models are present, IR must include processes to disable tools or isolate action pathways rapidly.
- AI incidents often involve complicated chain-of-custody issues. Enterprises should define how AI artifacts (e.g., logs, training data, embeddings) are preserved as evidence.

Control 18: Penetration Testing

Test the effectiveness and resiliency of enterprise assets through identifying and exploiting weaknesses in controls (people, processes, and technology), and simulating the objectives and actions of an attacker.

AI LLM Applicability

Penetration testing for AI systems must account for vulnerabilities and behaviors that do not exist in traditional applications. LLM-enabled systems involve model logic, data pipelines, orchestration layers, and tool integrations that require specialized testing techniques. Threats may originate not from malicious executable code but from natural-language prompts, adversarial text, poisoned documents, or high-volume querying.

Unique penetration testing considerations include:

- Prompt injection and policy bypass, where untrusted input alters system instructions or triggers unsafe actions
- Model extraction attempts, aiming to replicate or infer model parameters, training data, or proprietary behavior
- Testing guardrails and safety filters, probing whether safety controls can be evaded through adversarial phrasing
- RAG poisoning and indirect prompt manipulation, where malicious documents contaminate retrieval pipelines
- Fine-tuning misuse, including attempts to influence model behavior through small, targeted training data modifications
- Unsafe tool invocation, especially when LLMs can call APIs, execute code, perform browsing, or interact with internal systems
- Behavioral drift detection, validating whether model behavior after updates or retraining remains within safe boundaries

Since attackers can manipulate inputs rather than code, penetration testing for LLM-based applications requires new methodologies, specialized expertise, and closer integration with data science and AI engineering teams.

Safeguards

CIS Control 18: Penetration Testing

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
18.1	Establish and Maintain a Penetration Testing Program	Establish and maintain a penetration testing program appropriate to the size, complexity, industry, and maturity of the enterprise. Penetration testing program characteristics include scope, such as network, web application, Application Programming Interface (API), hosted services, and physical premise controls; frequency; limitations, such as acceptable hours, and excluded attack types; point of contact information; remediation, such as how findings will be routed internally; and retrospective requirements.		●	●	Ensure that penetration testing includes all AI components, including model interfaces, pipelines, integrations, and associated application logic. Testing should cover both backend components (model-serving APIs, vector stores, orchestration layers) and user-facing integrations to ensure that full life cycle exposure is assessed. The following should be in scope: ▪ Model gateway/API ▪ RAG ingestion and retrieval ▪ Vector database access controls ▪ Tool execution systems ▪ Prompt/policy management ▪ Tenant/admin consoles for SaaS models
18.2	Perform Periodic External Penetration Tests	Perform periodic external penetration tests based on program requirements, no less than annually. External penetration testing must include enterprise and environmental reconnaissance to detect exploitable information. Penetration testing requires specialized skills and experience and must be conducted through a qualified party. The testing may be clear box or opaque box.		●	●	Test for AI-specific attack patterns such as prompt injection, jailbreaks, model extraction, data leakage, and misuse of model capabilities. Pen testers should actively attempt to break guardrails, generate unsafe output, trigger unapproved tools, or extract sensitive model or data elements.

CIS Control 18: Penetration Testing

Safeguard	Title	Description	IG1	IG2	IG3	AI LLM Applicability
18.3	Remediate Penetration Test Findings	Remediate penetration test findings based on the enterprise's documented vulnerability remediation process. This should include determining a timeline and level of effort based on the impact and prioritization of each identified finding.		●	●	Prioritize and remediate findings from AI-specific penetration tests and red team exercises. Addressing identified weaknesses prevents real-world attackers from exploiting the same vulnerabilities to compromise AI systems or extract sensitive data.
18.4	Validate Security Measures	Validate security measures after each penetration test. If deemed necessary, modify rulesets and capabilities to detect the techniques used during testing.			●	Validate that monitoring, alerting, and incident response procedures correctly detect and respond to AI-specific attacks. Penetration tests should confirm that model extraction attempts, injection attacks, or poisoning behaviors generate appropriate telemetry and that incident response teams can act quickly.
18.5	Perform Periodic Internal Penetration Tests	Perform periodic internal penetration tests based on program requirements, no less than annually. The testing may be clear box or opaque box.			●	Conduct internal penetration tests targeting AI infrastructure and models to identify insider threat vectors. Verifies the effectiveness of internal controls and segmentation, ensuring that lateral movement to AI assets is difficult even for attackers inside the network.

Model Hosting and Deployment Considerations

Endpoint-Hosted Models

- Local LLM runtimes should be tested for risks arising from developer experimentation, such as insecure tool configurations or embedded secrets.
- Pen testers should examine whether local model files can be tampered with to alter behavior or introduce malicious payloads.
- Browsing or code-execution tools enabled on endpoints should be validated for sandbox bypass or unsafe outbound traffic.

Enterprise-Hosted Models

- Model-serving APIs should undergo targeted attack simulations (e.g., high-volume extraction attempts, jailbreak payloads, malformed inputs).
- Training and fine-tuning pipelines require testing for poisoning vectors, unauthorized script execution, and supply-chain risks.
- Vector stores, retrieval pipelines, and memory systems must be validated for poisoning resistance, privilege separation, and secure access boundaries.
- Orchestration systems (e.g., routers, gateways, agent frameworks) require end-to-end testing to confirm policies cannot be bypassed through model behavior.

SaaS-Hosted Models Accessed via API

- Penetration testing should validate whether SaaS integration points (e.g., API gateways, client libraries, downstream systems) correctly enforce enterprise controls.
- Testing should include adversarial prompts to evaluate how provider-side safety systems interact with enterprise-side guardrails.
- Enterprises must test integration flows for unanticipated model version changes, unsafe outputs, or inconsistent responses resulting from provider updates.
- Tests should confirm that compromised or over-permissioned API keys cannot be used to exceed intended capabilities.

Additional AI LLM Considerations

- Standard penetration tests should include AI red-teaming, using adversarial text crafted to probe model weaknesses. For model-level safety evaluation especially, internal red-teaming becomes much more significant than classical penetration testing because internal red teams will have much more knowledge of context, data content, intent, and other factors that can be used to exploit LLMs.
- Model-level vulnerabilities (e.g., susceptibility to specific jailbreak families) should be retested whenever model versions change.
- Tools invoked by LLMs must be tested like any high-privilege API: ensuring strong authorization, sandboxing, and non-reliance on natural-language validation.
- Testing must include multi-turn attacks, where the adversary builds malicious context over several prompt-response cycles.
- Pen testers should attempt to exploit the “glue code” between the model and the application (e.g., routing logic, RAG pipelines, memory systems), as these are frequent weak points.
- End-to-end security validation should ensure that potentially harmful model behaviors do not propagate downstream into production systems or user-facing interfaces.

Conclusion

As AI and Large Language Models become integrated into a wide range of enterprise processes, it is essential that their deployment and governance be anchored in established cybersecurity best practices. The CIS Critical Security Controls offer a structured and widely adopted framework that helps translate traditional Safeguards, such as asset inventory, access control, secure configuration, and logging, into clear expectations for managing the unique risks associated with LLMs. By aligning LLM infrastructure, data flows, and operational workflows with these Controls, enterprises can systematically address concerns such as unauthorized access, data exposure, and inappropriate model behavior. This alignment ensures that the technical foundations supporting AI systems are managed with the same rigor and consistency applied to other critical enterprise technologies.

More broadly, applying the CIS Controls to AI and LLM ecosystems reinforces the principle that responsible adoption requires both technical hardening and continuous oversight. Protecting the data used to train and prompt models, maintaining strong authentication around model interfaces, monitoring for anomalous activity, and evaluating model outputs for reliability all contribute to a comprehensive security posture. This approach supports the safe and predictable use of LLMs while preserving their potential to enhance efficiency and decision-making. By integrating established cybersecurity priorities with modern AI capabilities, enterprises can foster an environment in which innovation proceeds alongside disciplined, well-governed risk management.

Appendix A: CIS Controls

The CIS Critical Security Controls® (CIS Controls®) are a prioritized set of actions which collectively form a defense-in-depth set of best practices that mitigate the most common attacks against systems and networks. They are developed by a community of information technology (IT) experts who apply their first-hand experience as cyber defenders to create these globally accepted security best practices. The experts who develop the CIS Controls come from a wide range of sectors, including retail, manufacturing, healthcare, education, government, defense, and others. It is important to note that while the CIS Controls address general best practices that enterprises should implement to protect their environment, some operational environments may present unique requirements not addressed by the CIS Controls or require deviations from best practices.

Implementation Groups

The Implementation Group methodology was developed as a new way to prioritize the CIS Controls. These IGs provide a simple and accessible way to help enterprises of different classes focus their scarce security resources, while still leveraging the value of the CIS Controls program, community, and complementary tools and working aids. More about the Implementation Groups can be found in our [Guide to Implementation Groups \(IG\): CIS Critical Security Controls v8.1](#).

IG1

An IG1 enterprise is small to medium-sized with limited IT and cybersecurity expertise to dedicate toward protecting IT assets and personnel. The principal concern of these enterprises is to keep the business operational, as they have a limited tolerance for downtime. The sensitivity of the data that they are trying to protect is low and principally surrounds employee and financial information. Safeguards selected for IG1 should be implementable with limited cybersecurity expertise and aimed to thwart general, non-targeted attacks. These Safeguards will also typically be designed to work in conjunction with small or home office commercial off-the-shelf (COTS) hardware and software.

IG2

An IG2 enterprise employs individuals responsible for managing and protecting IT infrastructure. These enterprises support multiple departments with differing risk profiles based on job function and mission. Small enterprise units may have regulatory compliance burdens. IG2 enterprises often store and process sensitive client or enterprise information and can withstand short interruptions of service. A major concern is loss of public confidence if a breach occurs. Safeguards selected for IG2 help security teams cope with increased operational complexity. Some Safeguards will depend on enterprise-grade technology and specialized expertise to properly install and configure.

IG3

An IG3 enterprise employs security experts that specialize in the different facets of cybersecurity (e.g., risk management, penetration testing, application security). IG3 assets and data contain sensitive information or functions that are subject to regulatory and compliance oversight. An IG3 enterprise must address availability of services and the confidentiality and integrity of sensitive data. Successful attacks can cause significant harm to the public welfare. Safeguards selected for IG3 must abate targeted attacks from a sophisticated adversary and reduce the impact of zero-day attacks.

If you would like to know more about how the CIS Controls and Implementation Groups pertain to enterprises of all sizes, visit our website at <https://www.cisecurity.org/controls/cis-controls-list/>.

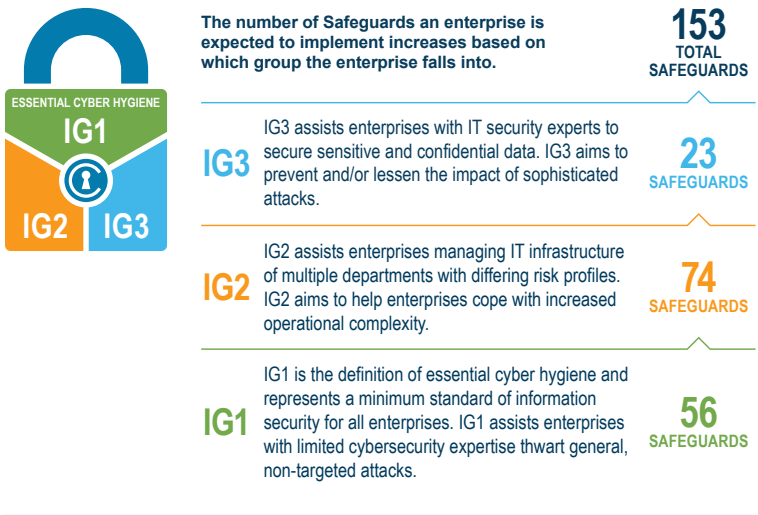


Figure 1: CIS Controls v8.1 Implementation Group levels.

Appendix B: Acronyms and Abbreviations

AAA	Authentication, Authorization, and Auditing
AI	Artificial Intelligence
AI LLM	Artificial Intelligence/Large Language Model
API	Application Programming Interface
API/SDK	Application Programming Interface/Software Development Kit
BEC	Business Email Compromise
BOM	Bill of Materials
BYOM	Bring Your Own Model
CA	Certificate Authority
CASB	Cloud Access Security Broker
chatML	Chat Markup Language
CI/CD	Continuous Integration/Continuous Delivery
CIS	Center for Internet Security
CIS CDM	CIS Community Defense Model
CIS CSAT	CIS Controls Self Assessment Tool
CIS RAM	CIS Risk Assessment Method
CMDB	Configuration Management Database
CMMC	Cybersecurity Maturity Model Certification
CPU	Central Processing Units
CRM	Customer Relationship Management
CUDA	Compute Unified Device Architecture
CVE	Common Vulnerabilities and Exposures
DHCP	Dynamic Host Configuration Protocol
DLP	Data Loss Prevention
DNS	Domain Name System
EDR	Endpoint Detection and Response
ERP	Enterprise Resource Planning
GPO	Group Policy Object
GPU	Graphics Processing Unit
HITL	Human-in-the-loop
HTML	HyperText Markup Language
HTTP/ HTTPS	Hypertext Transfer Protocol (Secure)
IAC	Infrastructure as Code
IAM	Identity and Access Management
IDE	Integrated Development Environment
IDS	Intrusion Detection System
IG	Implementation Group
IG1	Implementation Group 1
IG2	Implementation Group 2

IG3	Implementation Group 3
IP	Internet Protocol
IPS	Intrusion Prevention System
IR	Incident Response
ISO	International Organization for Standardization
IT	Information Technology
JSON	JavaScript Object Notation
LLM	Large Language Models
MCP	Model Context Protocol
MDM	Mobile Device Management
MFA	Multi-Factor Authentication
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
ML	Machine Learning
MNPI	Material Non-Public Information
NIST SP	National Institute of Standards and Technology Special Publication
OAuth	Open Authorization
OS	Operating System
PCI DSS	Payment Card Industry Data Security Standard
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
SDK	Software Development Kit
SDLC	Software Development Lifecycle
SecOps	Security Operations
SIEM	Security Information and Event Management
SLA	Service Level Agreement
SLM	Small Language Model
SOC 2	System and Organization Controls 2
SQL	Structured Query Language
SSO	Single Sign-On
SSRF	Server-Side Request Forgery
SWG	Secure Web Gateway
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
vLLM	Virtual Large Language Model
VM	Virtual Machine
VPC	Virtual Private Cloud
XML	Extensible Markup Language

Appendix C: Links and Resources

- **CIS Critical Security Controls (CIS Controls) v8.1:** Learn more about the CIS Controls, including how to get started, why each Control is critical, procedures and tools to use during implementation, and a complete listing of Safeguards for each Control.
- **CIS Controls Policy Templates:** Policy templates geared toward Safeguards found in IG1 of the CIS Controls.
- **A Roadmap to the CIS Controls:** There is a broader ecosystem that surrounds the CIS Controls that offers guidance, tools, resources, mappings, and more to help facilitate the adoption and implementation of the framework. This guide will help adopters understand what is available to them, where to start, and how to put it all together.
- **Establishing Essential Cyber Hygiene:** IG1 is essential cyber hygiene and represents a minimum standard of information security for all enterprises. This guide will help enterprises establish essential cyber hygiene.
- **Guide to Asset Classes:** In v8.1, CIS restructured Asset Classes and their respective definitions to ensure consistency throughout the Controls. Learn more about our naming conventions and what they mean.
- **Guide to Implementation Groups (IG):** IGs are the recommended guidance to prioritize implementation of the CIS Controls. In an effort to assist enterprises of every size, IGs are divided into three groups. Learn more about the five factors that can impact IG selection for an enterprise.
- **CIS Controls Assessment Specification:** Provides an understanding of what should be measured in order to verify that the Safeguards are properly implemented.
- **CIS Controls Navigator:** Learn more about the Controls and Safeguards and see how they map to other security standards (e.g., CMMC, NIST SP 800-53 Rev. 5, PCI DSS, MITRE ATT&CK). Available for CIS Controls versions 8.1, 8, and 7.1.
- **CIS Community Defense Model (CDM) v2.0:** A guide published by CIS that leverages the open availability of comprehensive summaries of attacks and security incidents, and the industry-endorsed ecosystem that is developing around the MITRE ATT&CK Framework.
- **CIS Risk Assessment Method (CIS RAM) v2.2:** An information security risk assessment method that helps enterprises implement and assess their security posture against the CIS Controls.
- **CIS SecureSuite® Membership:** Membership with access to CIS-CAT Pro Assessor, CIS Build Kits, CIS Benchmarks, and more.
- **CIS Benchmarks®:** Secure configuration guidelines for 100+ technologies, including operating systems, applications, and network devices.
- **CIS SecureSuite Platform:** A unified platform for CIS SecureSuite Members that provides organizations with the ability to assess their cybersecurity posture against the CIS Critical Security Controls® (CIS Controls®) and to demonstrate conformance with the CIS Benchmarks®.
- **CIS Build Kits:** ZIP files that contain a Group Policy Object (GPO) for each profile within the corresponding CIS Benchmark.
- **CIS Hardened Images®:** Virtual machine images securely pre-configured to the CIS Benchmarks.
- **CIS WorkBench:** Get involved in one of our many communities.
- **CIS Password Policy Guide:** CIS guidance for secure usage of passwords in an enterprise.

The Center for Internet Security, Inc. (CIS®) makes the connected world a safer place for people, businesses, and governments through our core competencies of collaboration and innovation. We are a community-driven nonprofit, responsible for the CIS Critical Security Controls® and CIS Benchmarks®, globally recognized best practices for securing IT systems and data. We lead a global community of IT professionals to continuously evolve these standards and provide products and services to proactively safeguard against emerging threats. Our CIS Hardened Images® provide secure, on-demand, scalable computing environments in the cloud. CIS is home to the Multi-State Information Sharing and Analysis Center® (MS-ISAC®), the trusted resource for cyber threat prevention, protection, response, and recovery for U.S. State, Local, Tribal, and Territorial government entities, and the Elections Infrastructure Information Sharing and Analysis Center® (EI-ISAC®), which supports the rapidly changing cybersecurity needs of U.S. election offices. To learn more, visit <http://cisecurity.org> or follow us on X: [@CISecurity](https://twitter.com/CISecurity).

-  cisecurity.org
-  email@cisecurity.org
-  518-266-3460
-  Center for Internet Security
-  @CISecurity
-  CenterforIntSec
-  cisecurity
-  TheCISecurity

